Learning Reservoir Operating Rules with Reinforcement Learning: a Tutorial with OpenHydroQual as the Simulator

OpenHydroQual-RL working notes

November 1, 2026

Abstract

This report teaches reinforcement learning (RL) through a worked water resources example. RL is a way of *learning an operating rule by trial and error in a simulator*. Here the simulator is an OpenHydroQual (OHQ) model, driven from Python, and the learner is a small neural network trained with PyTorch. We start with a single water-supply reservoir hit by random storms. Each day the learned rule chooses a release that must balance meeting a downstream demand against flooding the river and spilling water. We then move to a harder problem: two reservoirs in series with an uncontrolled tributary. Along the way we explain the RL concepts from the ground up, relating them to stochastic dynamic programming and model calibration. We also document the engineering: how the OHQ model is built from Python, how its output was verified against an exact solution (which uncovered and led to fixing two issues in OHQ), how training is parallelized, and how the learned rules compare with classical rule curves. No prior knowledge of RL is assumed; familiarity with hydrologic modeling and basic calculus and probability is.

How to read this report. Section 2 is a self-contained introduction to RL. Its first half is conceptual, and its last part (Section 2.5) gives the mathematics. Sections 3 to 5 set up the first problem and the simulator. Section 6 explains the learning algorithm (PPO) step by step. Section 7 shows the results. Section 9 repeats everything for the harder two-reservoir problem. A nomenclature list is at the front and an RL glossary at the end.

Contents

Nomenclature **4**

1 Overview **7**

2 Reinforcement learning in a nutshell **7**

2.1 What problem does RL solve? 7
2.2 The agent–environment loop 8
2.3 The policy 9
2.4 How learning works, in one paragraph 9
2.5 The same ideas in equations 10
2.6 Training versus testing 12
2.7 What the agent sees in this report 12

3 The first problem: a single reservoir **12**
3.1 System . . . 12
3.2 Stochastic forcing . . . 12
3.3 MDP formulation . . . 13
3.4 The objective function . . . 14

4 Building the OHQ model from Python **14**
4.1 Python bindings . . . 14
4.2 Components . . . 14
4.3 One control interval . . . 15

5 Verifying the simulator output **16**
5.1 Finding: one-step time-stamp lag . . . 16
5.2 Episode mass balance . . . 16

6 The RL algorithm (PPO) **16**
6.1 The idea . . . 17
6.2 The networks . . . 18
6.3 Step 2 in detail: generalized advantage estimation . . . 18
6.4 Step 3 in detail: the clipped objective . . . 18
6.5 What the two networks learn from . . . 19
6.6 The full training loop . . . 20
6.7 Inside PyTorch: how the actor is built and how its weights change . . . 20
6.8 Hyperparameters and what they control . . . 24
6.9 Two optional additions used for the stormwater pond . . . 25

7 Results for the single reservoir **25**
7.1 How we judge a policy . . . 25
7.2 Training . . . 26
7.3 Comparison with baselines . . . 26

8 Performance and the case for a codegen backend **28**

9 A harder problem – a two-reservoir cascade **31**
9.1 System . . . 31
9.2 OHQ model . . . 32
9.3 MDP and objective . . . 32
9.4 Baseline: a coordinated two-threshold rule . . . 33
9.5 Training . . . 34
9.6 Results . . . 35

10 A third problem – real-time control of a stormwater pond **36**
10.1 System and data . . . 40
10.2 Measuring water age with the Laplace tracer . . . 41
10.3 OHQ model and checks . . . 41
10.4 Objectives, reward and the exceedance problem . . . 42
10.5 Baselines . . . 43
10.6 Training . . . 44
10.7 Results . . . 44

11 Limitations and next steps **47**

12 How to reproduce **47**

Appendix: RL glossary **52**

Nomenclature

Units: volumes in m^3, flows in m^3/d, time in days. Because each decision step is one day, a volume over a step (m^3) and the mean flow over that step (m^3/d) are numerically equal; this is why volumes are compared directly with Q_{safe} and D_t.

Symbol	Meaning	Value / unit
Time and indices		
t	time; as a subscript, the day index of a decision step covering $[t, t+1]$	d
T	episode (season) length	90 steps
t_0	start time of one OHQ simulation interval	d
k	forecast lead time	1, 2, 3 d
j	storm index	–
τ	time since the onset of a storm, $t - t_j$	d
Δt_0	OHQ initial (and, in the environment, fixed) time step, `initial_time_step`	0.05 d
Δt	decision interval (one day)	1 d
Reservoir and structures		
$S(t)$, S_t	reservoir storage (continuous; at the start of day t)	m^3
S_0	initial storage of an episode, $\sim U(0.3, 0.8)S_{\text{cap}}$; also of a single simulation interval	m^3
S_{cap}	storage at the spillway crest (capacity)	50,000 m^3
S_{dead}	storage below which the outlet release tapers to zero	200 m^3
Q_t	release requested by the policy for day t	m^3/d
Q_{out}	outlet capacity (maximum release)	6,000 m^3/d
Q_{safe}	safe capacity of the downstream channel	3,000 m^3/d
$Q_{\min}, Q_{\max}$, $S_{\min}, S_{\max}$	parameters of the OHQ `Reservoir_link_rule` flow rule (Section 4)	–
$Q(S)$	flow of a `Reservoir_link_rule` link as a function of storage	m^3/d
$V_t^{in}, V_t^{rel}, V_t^{sp}$	inflow, released and spilled volume during day t	m^3
Forcing		
$Q_{in}(t)$	reservoir inflow	m^3/d
q_b	mean base flow	600 m^3/d
β	per-episode base-flow factor, $\sim U(0.7, 1.3)$	–
$h_j(t)$	hydrograph of storm j	m^3/d
t_j	onset time of storm j (probability 0.06 per day)	d
P_j	peak flow of storm j, $\sim U(5000, 15000)$	m^3/d
t_r	storm rise time	0.5 d
k_j	recession constant of storm j, $\sim U(0.3, 0.7)$	d^{-1}
$D(t)$, D_t	downstream demand (on day t)	m^3/d
$\bar{D}$	mean demand	1,500 m^3/d
ψ	demand phase, $\sim U(0, 2\pi)$	rad
$\hat{F}_{t+k}$	forecast, issued on day t, of the daily-mean inflow on day $t+k$	m^3/d
σ_F	log-normal forecast error scale, $0.2\sqrt{k}$	–

Symbol	Meaning	Value / unit
ξ	all random elements of a season (storms, β, ψ, forecast errors, S_0)	–
MDP, reward and objective		
o_t	observation (7 normalized features, Section 3.3)	–
a_t	action in $[-1, 1]$; $Q_t = \frac{1}{2}(a_t + 1)Q_{\text{out}}$	–
r_t	reward of day t	–
r_T^{term}	terminal (end-of-season storage) reward	–
w_f, w_s, w_T	weights of the flood, spill and terminal penalties	2, 1, 5
$J(\pi)$	objective: expected undiscounted season return, Eq. (8)	–
$J_\gamma(\pi)$	discounted return maximized during training	–
η	flood-rule threshold of the rule-curve baselines (fraction of S_{cap})	0.8; tuned 0.4
PPO		
$\pi_\theta(a \mid o)$	stochastic (Gaussian) policy with network parameters θ	–
$\mu_\theta(o)$	mean of the policy (actor network output)	–
σ	standard deviation of the policy (learned, state-independent)	$0.59 \to 0.15$
$V_\phi(o)$	value function (critic network) with parameters ϕ	–
γ	discount factor	0.99
λ	GAE parameter	0.95
δ_t	temporal-difference error	–
$\hat{A}_t$	advantage estimate (GAE), normalized per update	–
$\hat{R}_t$	return target for the critic, $\hat{A}_t + V_\phi(o_t)$	–
d_t	episode-end flag (1 on the last step of a season)	0/1
ρ_t	probability ratio of the new to the old policy	–
ϵ	PPO clipping parameter	0.2
c_v	value-loss coefficient	0.5
$L(\theta)$	PPO loss minimized by gradient descent	–
Two-reservoir cascade (Section 9)		
S_t^A, S_t^B	storage of reservoir A (upstream) and B (downstream)	m^3
S_{cap}^A, S_{cap}^B	capacities (spillway crests) of A and B	60,000; 30,000 m^3
S_n^A	storage of A at the end of solver step n within a day	m^3
Q_t^A, Q_t^B	releases requested by the policy for A and B	m^3/d
Q_{out}^A, Q_{out}^B	outlet capacities of A and B	6,000; 8,000 m^3/d
$Q_{in}^A, Q_{in}^B, Q^{tr}$	inflow to A, to B, and tributary flow	m^3/d
q_t^B, q_t^{tr}	current (start-of-day) inflow to B and tributary flow	m^3/d
$\bar{P}_c$	storm peak scale of catchment c	m^3/d
m	intensity of a regional storm, common to all catchments, $\sim U(0.5, 1.5)$	–
$V_t^{rel,A}, V_t^{sp,A}$	released and spilled volume of A during day t (enter B)	m^3
$V_t^{rel,B}, V_t^{sp,B}$	released and spilled volume of B during day t	m^3
V_t^{tr}	tributary volume during day t	m^3
V_t^{riv}	river volume at the flood point, $V_t^{rel,B} + V_t^{sp,B} + V_t^{tr}$	m^3

Symbol	Meaning	Value / unit
Q_t^{sup}	part of the demand the tributary does not supply (rule)	m^3/d
η_A, η_B	thresholds of the cascade rule (fractions of S_{cap}^A, S_{cap}^B)	–
Stormwater pond (Section 10)		
h	pond depth	m
α, β	pond storage–depth parameters, $S \approx \alpha h^\beta$	250; 2
u_t	valve opening chosen by the policy for hour t	0–1
C_d, A_{max}	valve discharge coefficient; orifice area at full opening	0.6; 0.03 m^2
ε	regularization head of the valve law	1 mm
$Q(t)$, Q_n	pond outflow (valve + overflow); at solver step n	m^3/d
δt	OHQ solver step (5 minutes)	d
Q_p	outflow exceeded a fraction p of the time	m^3/d
p	exceedance probability	0.001
λ	decay rate of the Laplace age tracer	1 d^{-1}
τ, $p(\tau)$	water age and its probability density	d
C	age-tracer concentration, $\mathcal{L}\{p\}(\lambda) = E[e^{-\lambda\tau}]$	–
F_y	young-water fraction of the outflow (flow-weighted C)	–
J	pond evaluation objective, $Q_p/Q_p^{passive} + F_y/F_y^{passive}$ (lower is better; passive pond = 2)	–
$Q_p^{passive}, F_y^{passive}$	Q_p and F_y of the passive pond (valve fully open) under the same rain	–
a_t (pond)	network action; valve opening $u_t = ((a_t + 1)/2)^2$	–
Q_{ref}, V_{ref}	reference flow and volume of the pond reward	1,000 m^3/d; 1,000 m^3
w_q, w_y	weights of the peak-flow surrogate and the young-water terms	1; 5
u_{hold}, u_{pre}	openings of the forecast rule (holding; pre-release)	–
Other		
$U(a, b)$	uniform distribution on $[a, b]$	–
$\mathcal{N}(\mu, \sigma^2)$	normal distribution	–
$\mathbb{E}$	expectation	–

1 Overview

Figure 1 shows the software stack. The RL agent (PyTorch) talks to a Gymnasium environment through the standard `reset/step` API. The environment generates the hydrologic forcing, computes the reward and delegates the physics to a *simulator backend*. The only backend implemented so far, `OHQBackend`, rebuilds and solves an OHQ model for each one-day control interval through `openhydroqual_py`, a pybind11 module that links `libOHQLib`. The backend interface is deliberately narrow so that a codegen-compiled OHQ kernel can later replace it (Section 8).

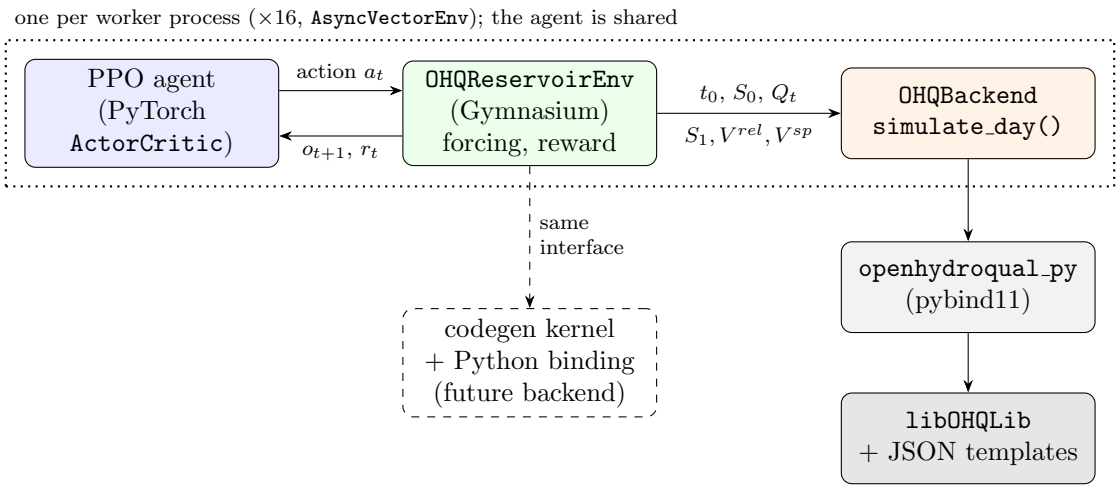


Figure 1: Software architecture. Each vectorized worker process owns one environment, one backend and its own OHQ working folder.

Files.

- `ohq_reservoir_env.py` – `OHQBackend`, `OHQReservoirEnv`, rule-curve baseline.
- `train_ppo.py` – PPO with GAE in plain PyTorch, parallel rollouts.
- `evaluate.py` – comparison against baselines, figures and table.
- `diagnostics.py` – simulator verification and timing figures.
- `report/` – this document.

2 Reinforcement learning in a nutshell

This section explains the ideas behind reinforcement learning (RL) for a reader who knows hydrologic modeling and calibration but has not used RL. Everything later in the report builds on it. The glossary at the end of the report collects the terms.

2.1 What problem does RL solve?

Think of a reservoir operator. Every morning they look at the reservoir level, the current inflow and the weather forecast, and decide how much water to release today. Tomorrow they see the consequences and decide again. The goal is not to make each single day look good. It is to do well over the whole season: to meet demand, avoid floods and not waste water over the spillway.

The operator does not know the future storms, so a good decision now must hedge against many possible futures.

This is a *sequential decision problem under uncertainty*. RL is a family of methods that *learn a decision rule for such problems by trial and error*: they try decisions in a simulator, observe what happens and gradually improve the rule. In this report, the simulator is an OpenHydroQual model.

How this differs from calibration. In model calibration we search for *parameter values* that make a model fit observed data (for example with a genetic algorithm or Levenberg–Marquardt). In RL we search for a *decision rule*, called a *policy*: a function that maps what the operator sees to what the operator does. The rule is judged by how well it performs over many simulated futures, not by how well it fits data.

Why not just optimize the release schedule? One could fix a storm sequence and optimize the 90 daily releases for it. But real storms are unknown in advance. A schedule that is optimal for one storm sequence can be disastrous for another: it might release water ahead of a storm that never comes. What we need is a rule that *reacts* to what it observes (closed-loop control) instead of a fixed plan (open-loop control). Classical alternatives each have a catch:

- **Rule curves** are hand-designed rules with a few parameters (for example, "release extra water above 80% full"). They are simple and transparent but limited to the shape the designer chose.
- **Stochastic dynamic programming** finds the optimal rule exactly, but its cost explodes with the number of state variables (the "curse of dimensionality"). The cascade in Section 9 has 16 observed quantities, far beyond what it can handle.
- **Model predictive control** re-optimizes a plan every day over a forecast horizon. It needs a fast optimizer inside the loop and a model of forecast uncertainty.

RL instead learns a flexible rule (here a small neural network) directly from simulations. The price is that it needs a lot of them, hundreds of thousands of simulated days here, which is why simulator speed matters (Section 8).

2.2 The agent–environment loop

RL describes the problem as a loop between an *agent* (the decision maker) and an *environment* (everything else) (Figure 2). At each step t:

1. the agent receives an **observation** o_t, the information available for the decision (storage, inflow, forecasts, demand, ...);
2. it chooses an **action** a_t (here, today's release);
3. the environment advances one step (here, OHQ simulates one day) and returns a **reward** r_t, one number that scores how good that step was, together with the next observation o_{t+1}.

A sequence of steps from a start to an end is an **episode**; here one episode is one 90-day season with its own random storms. The sum of the rewards over an episode is the **return**. The agent's goal is to choose actions that make the return as large as possible *on average over all the seasons it might face*.

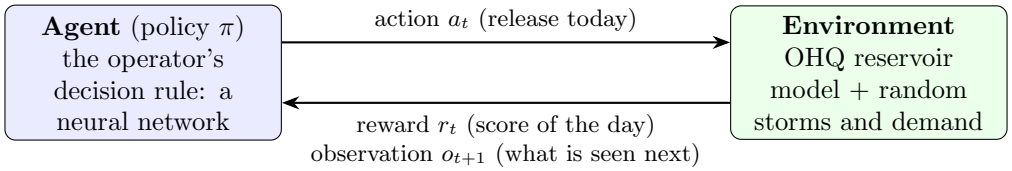


Figure 2: The agent–environment loop, repeated once per simulated day.

Designing the reward is designing the objective. The agent optimizes exactly what the reward measures and nothing else. Here the reward penalizes supply deficits, downstream flooding, spill and ending the season empty (Section 3.4). If a penalty were missing the agent would ignore that concern, and if a weight were too large it would over-react to it. Choosing the reward is the main place where engineering judgment enters.

Observation versus state. The *state* is everything that determines the future. The *observation* is what the agent actually sees. The agent never sees future storms, only noisy forecasts of them, so it must act well despite incomplete information, exactly like a real operator.

Discounting. Many RL methods maximize a *discounted* return, $r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$, with a discount factor γ slightly below 1 (here 0.99). Rewards far in the future count a little less. This makes learning more stable, because distant consequences are hard to attribute to today's action. The price is a mild preference for the near term (discussed in Section 3.4).

2.3 The policy

The **policy** π is the decision rule: given o_t, it returns a_t. A rule curve is a policy with one or two parameters. RL uses a much more flexible function, a small **neural network**. Here it is a stack of two layers of 64 simple nonlinear units, which can represent smooth functions of all the observed quantities at once. Its adjustable numbers, the *weights*, collectively written θ, play the role that model parameters play in calibration. Training an RL agent means adjusting θ.

During training the policy is **stochastic**: the network outputs a *mean* release, and the actual release is drawn from a bell curve around it. This random perturbation is how the agent **explores**, trying slightly different actions to discover which work better. The width of the bell curve, σ, is learned too, and it shrinks as the agent becomes confident (see the learning curves). When we *use* the trained policy for evaluation, we switch the randomness off and apply the mean action.

2.4 How learning works, in one paragraph

The agent plays many seasons in the simulator with its current policy and records what happened. For every decision it asks: *did things turn out better or worse than expected after this action?* It then adjusts θ so that actions that turned out better than expected become more likely, and actions that turned out worse become less likely. Then it plays more seasons with the improved policy, and so on. Two difficulties make this harder than it sounds:

- **Credit assignment.** A flood on day 50 may be the fault of a decision on day 40 not to release water. The agent must work out which earlier actions deserve the blame. It does this with a *value function* (next paragraph).

- **Noise.** Two seasons can differ wildly in storminess, so a bad outcome may be bad luck rather than a bad action. Comparing against "what was expected" filters out much of this luck.

The value function (the "critic"). Alongside the policy, a second neural network $V_\phi(o)$ learns to predict the return still to come from a given situation: roughly, "how good is it to be here?". It is like a forecast of the remaining season's score given today's storage, inflows and forecasts. Comparing what actually happened after an action with this prediction tells the agent whether the action was better or worse than expected. The policy network is then called the *actor* and the value network the *critic*, and the method an *actor–critic* method. Section 6 gives the details for the specific algorithm used, PPO.

2.5 The same ideas in equations

This subsection states the ideas above precisely. It assumes comfort with probability and calculus, and makes the link to stochastic dynamic programming (SDP), which many water resources engineers know from reservoir operation.

Markov decision process. The problem is formalized as a *Markov decision process* (MDP): states $s \in \mathcal{S}$, actions $a \in \mathcal{A}$, a transition law $p(s_{t+1} \mid s_t, a_t)$, a reward function $r_t = r(s_t, a_t, s_{t+1})$ and a horizon T. "Markov" means the next state depends only on the current state and action, not on the earlier history. In our problem the state contains the storage, the current and future storm hydrographs and the demand. The transition law is the OHQ mass balance plus the random weather. Crucially, we *never write p down*: we can only *sample* from it by running the simulator. This is the key practical difference from SDP, which needs p explicitly as transition probabilities on a discretized grid.

Policy and objective. A stochastic policy $\pi_\theta(a \mid o)$ is a probability density over actions given the observation, with parameters θ. In this report it is a normal density,

$$\pi_\theta(a \mid o) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\Big(- \frac{(a - \mu_\theta(o))^2}{2\sigma^2} \Big),$$

whose mean $\mu_\theta(o)$ is the output of the neural network. A *trajectory* $\tau = (o_0, a_0, r_0, o_1, a_1, r_1, \dots)$ is one simulated season, and its discounted return is $G(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t$. The goal is to find

$$\theta^\star = \arg\max_\theta \; J(\theta), \qquad J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\big[G(\tau)\big], \tag{1}$$

where the expectation is over the random weather *and* the policy's own randomness. This is an optimization over θ, just like calibration. The difference is that the objective is an expectation we can only estimate by simulating seasons.

Value functions and the Bellman equation. The *value* of a state under policy π is the expected return still to come,

$$V^\pi(s_t) = \mathbb{E}_\pi\Big[\sum_{k=0}^{T-1-t} \gamma^k r_{t+k} \;\Big|\; s_t \Big],$$

and the *action value* $Q^\pi(s,a)$ is the same quantity when action a is taken first and π is followed afterwards. Splitting off the first reward gives the *Bellman equation*

$$V^\pi(s_t) = \mathbb{E}\big[r_t + \gamma\, V^\pi(s_{t+1})\big], \qquad V^\star(s_t) = \max_a\ \mathbb{E}\big[r_t + \gamma\, V^\star(s_{t+1}) \;\big|\; s_t, a\big]. \tag{2}$$

The right-hand equation is exactly the backward recursion of SDP for reservoir operation: the best value today is the best immediate reward plus the expected (discounted) value of tomorrow's storage. SDP solves it by discretizing the state (for example storage into 50 classes and inflow into 10) and sweeping backward in time. This is exact but needs the transition probabilities, and the grid grows exponentially with the number of state variables. With the 16 observed quantities of the cascade problem, a grid of only 10 levels each would have 10^{16} cells. RL keeps the Bellman logic but (i) replaces the grid by neural networks that approximate V and π smoothly, and (ii) replaces the transition probabilities by simulated samples. In this sense RL is *approximate, simulation-based dynamic programming.*

The policy gradient. To improve θ by gradient ascent we need $\nabla_\theta J$, but J is an expectation over simulator runs, and the simulator (OHQ) is not differentiable. The *log-derivative trick* gets around this. Write the probability of a trajectory as $P_\theta(\tau) = p(s_0)\prod_t \pi_\theta(a_t \mid o_t)\, p(s_{t+1} \mid s_t, a_t)$. Then

$$\nabla_\theta J = \nabla_\theta \int P_\theta(\tau) G(\tau)\, d\tau = \int P_\theta(\tau)\, \nabla_\theta \log P_\theta(\tau)\, G(\tau)\, d\tau = \mathbb{E}_\tau\Big[G(\tau) \sum_t \nabla_\theta \log \pi_\theta(a_t \mid o_t)\Big],$$

because $\nabla P = P\, \nabla \log P$, and the transition terms $p(\cdot)$ do not depend on θ, so their log-derivatives vanish. *The physics drops out*: we only need to differentiate our own policy network, never the simulator. The expectation can be estimated by averaging over simulated seasons. In words: push up the log-probability of every action in a season in proportion to how good that season was.

Reducing the noise: baselines and advantages. That estimator is correct but extremely noisy: a stormy season punishes every action in it, good or bad. Two refinements fix most of this. (i) An action cannot affect rewards that came before it, so only the reward *after* step t is used to judge a_t. (ii) Subtracting any function of the observation, $b(o_t)$, does not change the expected gradient, because $\mathbb{E}_{a\sim\pi}[\nabla_\theta \log \pi_\theta(a \mid o)] = \nabla_\theta \int \pi_\theta\, da = \nabla_\theta 1 = 0$. But a good choice of b greatly reduces the variance. The best simple choice is the value function itself, which gives

$$\nabla_\theta J = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid o_t)\; A^\pi(o_t, a_t)\Big], \qquad A^\pi(o_t,a_t) = Q^\pi(o_t,a_t) - V^\pi(o_t). \tag{3}$$

The **advantage** A measures how much better action a_t was than what the policy does on average in that situation. This is the precise form of "did things turn out better or worse than expected?". Positive advantage: make that action more likely. Negative: less likely.

Estimating the advantage: the TD error. We do not know Q^π or V^π. We learn an approximation $V_\phi \approx V^\pi$ (the critic) by least-squares regression on observed returns. Then the one-step Bellman relation (2) gives a cheap advantage estimate, the *temporal-difference (TD) error*

$$\delta_t = r_t + \gamma V_\phi(o_{t+1}) - V_\phi(o_t). \tag{4}$$

Read it as "what actually happened today, plus what the critic now expects for the rest of the season, minus what the critic expected yesterday". If $\delta_t > 0$, the day went better than expected. Averaging it over many steps is how credit is assigned through time: Section 6 shows how PPO blends several steps of δ into a better estimate.

2.6 Training versus testing

As in calibration and validation, an agent must be judged on data it was not trained on. Each season's random storms are generated from a *seed* (a number that fixes the random sequence). Training uses one set of seeds. Testing uses 30 different seeds, the same 30 for every policy, so that all policies face identical storms and the comparison is fair.

2.7 What the agent sees in this report

RL concept	In the single-reservoir problem
environment	OHQ reservoir model + storm and demand generator
step	one day (one OHQ run of one day)
episode	one 90-day season
observation o_t	storage, current inflow, 1–3-day inflow forecasts, demand, days left
action a_t	release for the day, from 0 to 6,000 m^3/d
reward r_t	minus the day's penalties (deficit, flooding, spill)
return	minus the season's total penalty (closer to 0 is better)
policy	neural network mapping o_t to a_t

3 The first problem: a single reservoir

3.1 System

A single reservoir with spillway-crest storage $S_{\mathrm{cap}} = 50{,}000\ \mathrm{m}^3$ receives a low base flow plus random storm hydrographs (Figure 3). The operator controls the release Q_t through an outlet works with capacity $Q_{\mathrm{out}} = 6{,}000\ \mathrm{m}^3/\mathrm{d}$. The downstream channel floods when its flow exceeds $Q_{\mathrm{safe}} = 3{,}000\ \mathrm{m}^3/\mathrm{d}$. Water above S_{cap} leaves over an uncontrolled emergency spillway. Downstream users need a demand $D(t)$ with a mean of $1{,}500\ \mathrm{m}^3/\mathrm{d}$ and a $\pm 30\%$ seasonal swing, which exceeds the mean base flow ($\approx 600\ \mathrm{m}^3/\mathrm{d}$). Storage is therefore needed to bridge dry spells. Storms, on the other hand, can bring 10–35 thousand m^3 each.

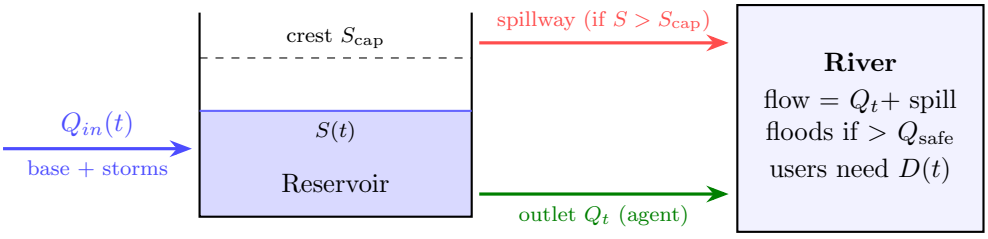


Figure 3: The toy reservoir system.

3.2 Stochastic forcing

For every episode (a 90-day season) the environment draws a new inflow series on a 0.05-day grid:

$$Q_{in}(t) = \beta\, q_b + \sum_j h_j(t), \qquad \beta \sim U(0.7, 1.3),\ q_b = 600\ \mathrm{m}^3/\mathrm{d}, \tag{5}$$

$$h_j(t) = P_j \begin{cases} \tau/t_r & 0 \le \tau < t_r \\ e^{-k_j(\tau - t_r)} & \tau \ge t_r \end{cases}, \quad \tau = t - t_j, \tag{6}$$

with storm onsets t_j arriving with probability 0.06 per day, peaks $P_j \sim U(5000, 15000)$ m^3/d, rise time $t_r = 0.5$ d and recession constants $k_j \sim U(0.3, 0.7)$ d^{-1}. The demand is $D(t) = \bar{D}\,[1 + 0.3\sin(2\pi t/T + \psi)]$ with a random phase $\psi \sim U(0, 2\pi)$. The agent also receives a noisy forecast of the daily-mean inflow for the next 1, 2 and 3 days, with multiplicative log-normal errors that grow with lead time ($\sigma_F = 0.2\sqrt{k}$ for lead k days). Figure 4 shows three realizations.

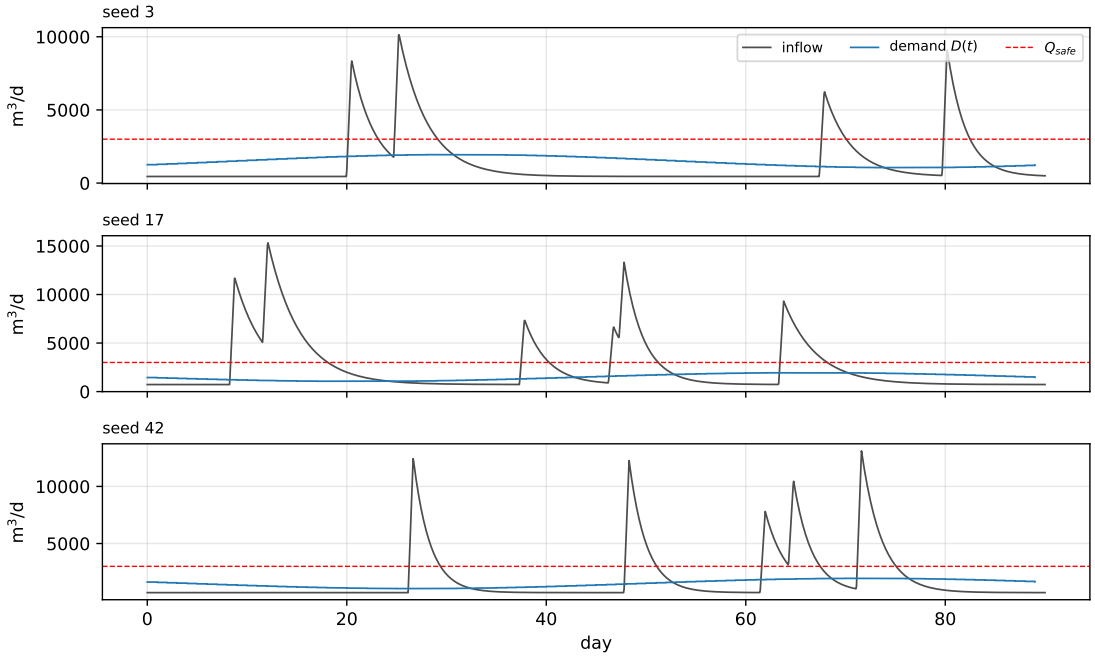


Figure 4: Three random forcing realizations (inflow, demand and the safe channel capacity).

3.3 MDP formulation

Decision step: one day; episode length $T = 90$ steps ($t = 0, \ldots, T-1$). The initial storage is drawn from $U(0.3, 0.8)\,S_{\text{cap}}$.

Observation (7-dim, normalized):

$$o_t = \big(S_t/S_{\text{cap}},\ Q_{in}(t)/Q_{\text{safe}},\ \hat{F}_{t+1}/Q_{\text{safe}},\ \hat{F}_{t+2}/Q_{\text{safe}},\ \hat{F}_{t+3}/Q_{\text{safe}},\ D_t/Q_{\text{safe}},\ 1 - t/90\big),$$

where $\hat{F}_{t+k}$ is the k-day-ahead forecast of the daily-mean inflow.

Action: $a_t \in [-1, 1]$, mapped to the target release $Q_t = \frac{1}{2}(a_t + 1)\,Q_{\text{out}}$.

Transition: one-day OHQ simulation (Section 4).

Reward: with V_t^{rel} and V_t^{sp} the released and spilled volumes over the day,

$$r_t = -\Big(\frac{\max(0, D_t - V_t^{rel})}{D_t}\Big)^2 - w_f\,\frac{\max(0, V_t^{rel} + V_t^{sp} - Q_{\text{safe}})}{Q_{\text{safe}}} - w_s\,\frac{V_t^{sp}}{Q_{\text{safe}}}, \tag{7}$$

with $w_f = 2$ and $w_s = 1$. At the end of the season a terminal penalty $r_T^{\text{term}} = -w_T \max(0,\, 0.5 S_{\text{cap}} - S_T)/S_{\text{cap}}$ ($w_T = 5$) discourages emptying the reservoir to avoid all flood risk.

3.4 The objective function

Let ξ denote everything random in a season: the storm onsets, peaks and recessions, the base-flow factor β, the demand phase ψ, the forecast errors and the initial storage S_0. An operating policy π maps the observation o_t to a release Q_t. The quantity we want to *maximize* is the expected season return

$$J(\pi) \;=\; \mathbb{E}_{\xi}\Big[\sum_{t=0}^{T-1} r_t \;+\; r_T^{\text{term}}\Big], \tag{8}$$

equivalently, we *minimize* the expected season cost

$$\mathbb{E}_{\xi}\Big[\sum_{t=0}^{T-1}\Big(\underbrace{\Big(\tfrac{\max(0,D_t-V_t^{rel})}{D_t}\Big)^2}_{\text{supply deficit}}+\underbrace{w_f\,\tfrac{\max(0,V_t^{rel}+V_t^{sp}-Q_{\text{safe}})}{Q_{\text{safe}}}}_{\text{downstream flooding}}+\underbrace{w_s\,\tfrac{V_t^{sp}}{Q_{\text{safe}}}}_{\text{uncontrolled spill}}\Big)+\underbrace{w_T\,\tfrac{\max(0,\,0.5S_{\text{cap}}-S_T)}{S_{\text{cap}}}}_{\text{end-of-season storage}}\Big],$$

subject to the reservoir dynamics simulated by OHQ, $S_{t+1} = S_t + V_t^{in} - V_t^{rel} - V_t^{sp}$, the outlet and spillway rules of Section 4, and $0 \le Q_t \le Q_{\text{out}}$. All terms are dimensionless, so the weights $w_f = 2$, $w_s = 1$, $w_T = 5$ set the trade-off: one day of flow 100% above Q_{safe} costs as much as two days of a total supply failure.

Two versions of this objective appear in the report:

- **Evaluation** (Section 7) uses Eq. (8) directly: the undiscounted return of each test season, averaged over 30 held-out seasons. This is the "return" in all tables and plots.
- **Training.** PPO maximizes the *discounted* return $J_\gamma(\pi_\theta) = \mathbb{E}\big[\sum_t \gamma^t r_t\big]$ with $\gamma = 0.99$, through the clipped surrogate $L(\theta)$ of Section 6. Over a 90-day season, $\gamma^{89} \approx 0.41$, so the training objective weights late-season penalties, including the terminal storage term, at roughly half their evaluation weight. This is a standard variance-reduction compromise. It may explain part of the late-season over-drawdown seen in Figure 11; training with γ closer to 1 is a cheap thing to try.

The baselines are not optimized by PPO: the tuned rule curve maximizes the same Eq. (8) by a one-dimensional grid search over its threshold η.

The trade-off is the classic one: a full reservoir secures supply but leaves no room to absorb storms, while an empty one does the opposite. The release needed to make room is itself limited by Q_{safe}, so the agent must pre-release *before* a forecast storm arrives.

4 Building the OHQ model from Python

4.1 Python bindings

The bindings (`OpenHydroQual/PythonBindings/bindings.cpp`) expose `System` with methods for loading templates, setting system values, creating blocks and links, `finalize`, `solve` and `write_outputs`. There is no in-memory accessor for output time series, so results are read back from the output file. Each worker therefore uses its own working folder.

4.2 Components

The model uses existing OHQ templates:

OHQ object	type (template)	role
`Reservoir`	`Reservoir`	storage S, inflow time series (`Reservoir_operation.json`)
`River`	`fixed_head`	sink; its storage counts the released volume (`main_components.json`)
`SpillSink`	`fixed_head`	sink; its storage counts the spilled volume
`outlet`	`Reservoir_link_rule`	controlled release
`spillway`	`Reservoir_link_rule`	emergency overflow

`Reservoir_link_rule` computes its flow from a piecewise-linear rule in storage:

$$Q(S) = \begin{cases} Q_{\min} & S < S_{\min} \\ Q_{\min} + \frac{S - S_{\min}}{S_{\max} - S_{\min}}(Q_{\max} - Q_{\min}) & S_{\min} \le S \le S_{\max} \\ Q_{\max} & S > S_{\max} \end{cases} \tag{9}$$

We use the same link type for both structures:

- **outlet:** $Q_{\min} = 0$, $Q_{\max} = Q_t$ (the agent's action), $S_{\min} = 0$, $S_{\max} = S_{dead} = 200$ m^3. The outlet releases exactly Q_t unless the reservoir is nearly empty, where the release tapers to zero, so storage can never become negative.
- **spillway:** $Q_{\min} = 0$, $S_{\min} = S_{\text{cap}}$, $S_{\max} = 1.05 S_{\text{cap}}$, $Q_{\max} = 50{,}000$ m^3/d, which acts as an overflow weir. An earlier, steeper ramp ($1.02 S_{\text{cap}}$, 2×10^5 m^3/d) made the Newton solver fail to converge when the spill switched off, so the ramp was softened.

4.3 One control interval

Because the bindings do not expose a way to change a block's state on a live `System`, the backend rebuilds the model for every decision step, using the current storage as the initial condition. The inflow file holds the whole season in absolute time, and each step simulates $[t_0, t_0 + 1]$:

```
s = ohq.System(); s.set_default_template_path(R); s.set_working_folder(W)
s.load_template(R + "main_components.json")
s.add_template(R + "Reservoir_operation.json")
for k, v in {"simulation_start_time": t0, "simulation_end_time": t0 + 1,
             "initial_time_step": 0.05, "max_timestep_increase_factor": 1,
             "n_threads": 1, "alloutputfile": W + "output.txt"}.items():
    s.set_system_value(k, str(v))
s.create_block("Reservoir", "Reservoir", {"Storage": S0, "inflow": inflow_file})
s.create_block("River", "fixed_head", {"head": "0", "Storage": "0"})
s.create_block("SpillSink", "fixed_head", {"head": "0", "Storage": "0"})
s.create_link("outlet", "Reservoir_link_rule", "Reservoir", "River",
              {"Q_min": 0, "Q_max": Q, "S_min": 0, "S_max": S_dead})
s.create_link("spillway", "Reservoir_link_rule", "Reservoir", "SpillSink",
              {"Q_min": 0, "Q_max": 5e4, "S_min": S_cap, "S_max": 1.05 * S_cap})
s.finalize(); s.solve(True, False); s.write_outputs()
# read S(t0+1), River storage (= released volume), SpillSink storage (= spill)
```

(Numbers are converted to strings in the actual code.) Two practical notes: `set_system_value` returned `False` even when the setting was applied (fixed in OHQ, see the last section), and the `Reservoir_link_rule` template stores its `"unit"` entry inside the `rule` dictionary, so OHQ prints harmless *"property 'ft' does not exist"* messages.

5 Verifying the simulator output

Before using OHQ inside an RL loop we checked it against a case with an analytical solution: constant inflow 1000 m^3/d, release 150 m^3/d and $S_0 = 1000$ m^3, so that $S(t) = 1000 + 850\,t$.

5.1 Finding: one-step time-stamp lag

The first test runs disagreed with the analytical solution, and the error grew with the time step. Reading `System::Solve` showed two causes:

1. The time loop runs `while (t < tend + dt)`, so the last step overshoots t_{end}. With the default adaptive step (allowed to grow to 50× the initial step), this overshoot can be large.

2. The current OHQ source stamps each output row with the time at the *end* of the step (`PopulateOutputs(true, t + dt)`), and a code comment notes this was changed. The compiled `libOHQLib.so` the bindings link against predates the change and stamps the row with the *start* time of the step. Each row therefore holds the state one step later than its label says.

Figure 5 shows the storage error relative to the analytical solution. With the step held fixed, reassigning each row the next row's stamp makes the error exactly zero. With the default growing step, the error stays offset even after this correction. The environment therefore (i) fixes the step (`max_timestep_increase_factor = 1`), (ii) reassigns each row the next row's stamp and interpolates at $t_0 + 1$, and (iii) detects automatically at start-up which convention the linked library uses, by running this analytical test. The environment keeps working if OHQLib is rebuilt with the corrected stamping.

Update (rebuilt library). After rebuilding `libOHQLib` (v2.1.1) and the bindings from the current source, the self-check selects the end-of-step convention. The output now matches the analytical solution exactly with both the fixed and the default growing step, and the first row holds the true initial state. The overshoot past t_{end} remains by design: with the growing step, the 1-day run above ended at $t = 1.8$, so up to 80% extra time was simulated. The environment therefore keeps the step fixed. Training results are unaffected, since the physics is identical (same episode return, to 4 decimals). Per-step cost dropped from about 5.1 to 4.0 ms.

5.2 Episode mass balance

Over a full episode, run with the rule-curve policy, we compare the daily change in storage with the exact inflow volume (trapezoidal integral of the forcing) minus the released and spilled volumes (Figure 6). The residual follows a characteristic $+/-$ pattern on storm days: positive on the rising limb, then negative and decaying. This is consistent with OHQ evaluating the piecewise-linear inflow series slightly offset within the implicit step, and it is a property of the time discretization, not of the bookkeeping. The residual integrates to nearly zero: the largest daily residual is 231 m^3, while the cumulative residual over the season is 9.3 m^3 out of 189,339 m^3 of inflow (0.005%).

6 The RL algorithm (PPO)

We use *Proximal Policy Optimization* (PPO) [1], one of the most widely used RL algorithms. It is an actor–critic method built directly on the policy gradient of Eq. (3). It is implemented from

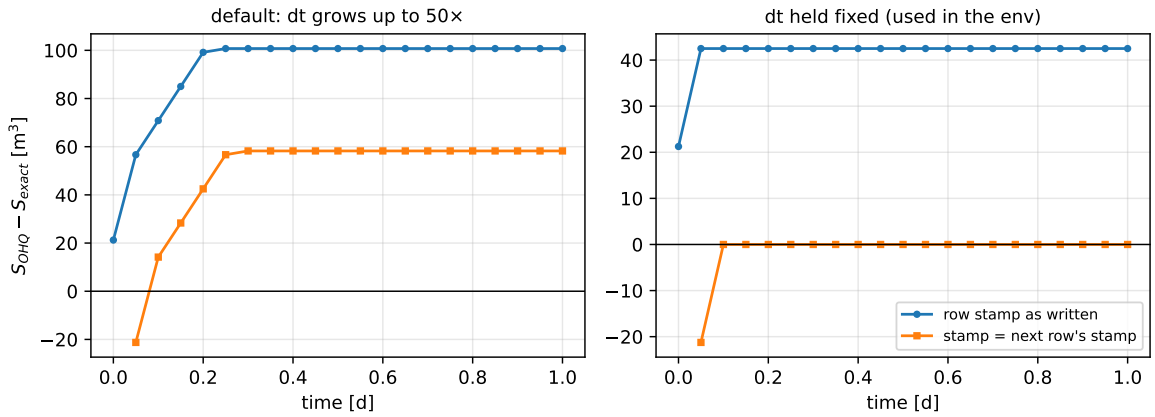


Figure 5: Error of the OHQ storage relative to the analytical solution for a one-day run (initial step 0.05 d). Left: default adaptive step. Right: fixed step, as used in the environment. Assigning each row the next row's stamp removes the error (after the very first row).

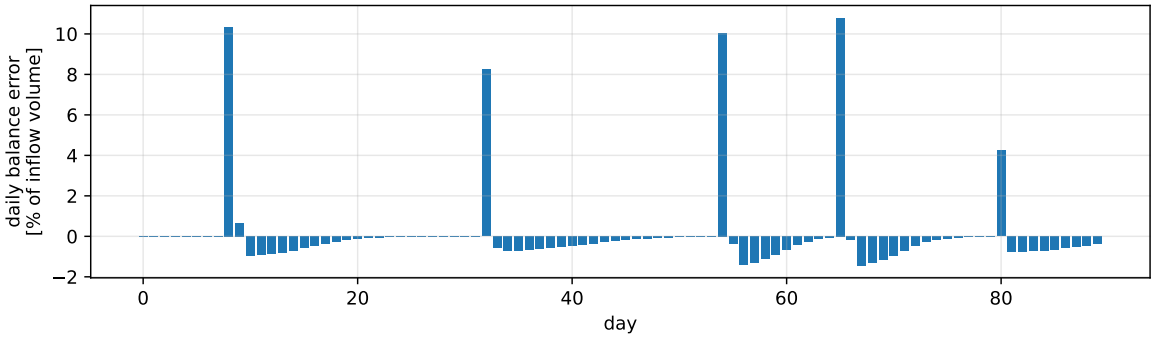


Figure 6: Daily mass-balance residual $S_t + V_t^{in} - V_t^{rel} - V_t^{sp} - S_{t+1}$ as a percentage of the day's inflow volume.

scratch in about 150 lines of PyTorch (`train_ppo.py`), so every step below can be traced in the code.

6.1 The idea

PPO repeats one cycle:

1. **Collect experience.** Run the current policy (with its exploration noise) in the simulator for a while and record every (o_t, a_t, r_t).
2. **Score each decision.** Use the critic to estimate each action's advantage: was it better or worse than expected?
3. **Improve the policy, carefully.** Change θ to make good actions more likely and bad ones less likely, *but not by too much in one go.*
4. **Improve the critic.** Fit V_ϕ to the returns just observed, so the next round's advantage estimates are better.

The phrase "not by too much" is what distinguishes PPO. A policy-gradient step uses data collected with the *old* policy, so it is only trustworthy for policies close to it. A large step can jump to a policy that behaves very differently, where the old data say nothing, and performance can collapse. Engineers will recognize the same concern behind step-size control in Newton's method or the damping term in Levenberg–Marquardt: trust the local model only within a limited region.

6.2 The networks

Two separate small networks are used. Each has two hidden layers of 64 units with tanh activations.

- **Actor:** $o_t \mapsto \mu_\theta(o_t)$, the mean of the action. The action is drawn as $a_t \sim \mathcal{N}(\mu_\theta(o_t), \sigma^2)$. The spread σ is one more learned parameter, initialized at $e^{-0.5} \approx 0.6$ (on an action range of $[-1, 1]$, so exploration starts wide). The environment clips the action to $[-1, 1]$ and maps it to a release.
- **Critic:** $o_t \mapsto V_\phi(o_t)$, the predicted (discounted) return still to come.

For evaluation, the trained actor is used deterministically: $a_t = \text{clip}(\mu_\theta(o_t))$.

6.3 Step 2 in detail: generalized advantage estimation

The TD error of Eq. (4) is a one-day look-ahead. It has low variance, but it is biased when the critic is wrong. The full remaining return is unbiased but noisy. *Generalized advantage estimation* (GAE) [2] blends all look-ahead lengths with a geometric weight $\lambda \in [0, 1]$:

$$\hat{A}_t = \sum_{l \geq 0} (\gamma\lambda)^l \, \delta_{t+l}, \qquad \delta_t = r_t + \gamma V_\phi(o_{t+1})(1 - d_t) - V_\phi(o_t), \tag{10}$$

where $d_t = 1$ on the last day of a season (there is no "tomorrow" to bootstrap from). $\lambda = 0$ gives the one-day TD error. $\lambda = 1$ gives the full remaining return minus the critic's prediction. We use $\lambda = 0.95$, so credit for a flood is spread back over roughly the previous $1/(1 - \gamma\lambda) \approx 17$ days. The sum is computed in one backward pass over the recorded data. The critic's regression target is $\hat{R}_t = \hat{A}_t + V_\phi(o_t)$. Before each update the advantages are standardized (zero mean, unit variance), which only rescales the step.

6.4 Step 3 in detail: the clipped objective

After the data are collected, the policy used to collect them is frozen as $\pi_{\theta_{old}}$. For a candidate θ, the *probability ratio*

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\theta_{old}}(a_t \mid o_t)}$$

says how much more (or less) likely the new policy is to take the recorded action. The objective $\mathbb{E}[\rho_t \hat{A}_t]$ has the same gradient as Eq. (3) at $\theta = \theta_{old}$, and it can be maximized with several gradient steps on the same data. Left alone, though, it would keep pushing ρ_t for good actions towards infinity. PPO therefore maximizes the *clipped* objective

$$L^{\text{clip}}(\theta) = \mathbb{E}_t\Big[\min\big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t,\, 1 - \epsilon,\, 1 + \epsilon)\, \hat{A}_t\big)\Big], \qquad \epsilon = 0.2. \tag{11}$$

Two cases make it concrete. If $\hat{A}_t > 0$ (a good action), the objective grows with ρ_t until $\rho_t = 1.2$ and is flat beyond, so there is no incentive to make that action more than 20% more likely in

one round. If $\hat{A}_t < 0$ (a bad action), the objective improves as ρ_t falls, but only until $\rho_t = 0.8$. The min makes sure clipping can only remove incentives, never add them. The result is a cheap, approximate trust region.

The quantity actually minimized by gradient descent combines the actor and critic losses:

$$L(\theta, \phi) = -L^{\text{clip}}(\theta) + c_v \, \tfrac{1}{2} \, \mathbb{E}_t\big[(V_\phi(o_t) - \hat{R}_t)^2\big], \qquad c_v = 0.5. \tag{12}$$

The first term improves the policy and the second fits the critic by least squares.

6.5 What the two networks learn from

Readers who know supervised learning usually ask: what are the inputs, and what are the labels? For PPO the answer differs for the two networks.

Inputs. Both networks receive *exactly the same input*: the observation o_t that the environment returns (for the pond, the 9 numbers of Section 10: pond depth, pond tracer concentration, current inflow, five rain forecasts, and the fraction of the episode left). Neither network sees the action as an input, nor anything the agent cannot observe (for the pond, e.g. the catch-basin state or the true future rain). Because the critic is used only during training, it could in principle be given extra, "privileged" information to sharpen its predictions. That is a known technique, but it is not used here.

The critic: a regression with self-made labels. The critic $V_\phi(o_t)$ outputs one number, the expected discounted reward still to come. Its label is the return estimate built from the rollout,

$$\hat{R}_t = \hat{A}_t + V_{\phi_{old}}(o_t) \;\approx\; r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots,$$

with $\hat{A}_t$ from Eq. (10). In words, the label sums the rewards that actually followed. Further ahead, the sum is cut off and the remainder is filled in with the critic's own prediction from when the data were collected ("bootstrapping"). GAE blends the cut-off over all horizons with weight $\lambda = 0.95$, so in effect about $1/(1-\gamma\lambda) \approx 17$ steps of actual rewards are used before the critic's own estimate takes over. At the end of an episode the future is zero. The critic is then fitted by least squares, $\frac{1}{2}(V_\phi(o_t) - \hat{R}_t)^2$, weighted by $c_v = 0.5$ in the loss. When reward normalization is on (the pond), the rewards in these labels are the normalized, clipped ones.

The actor: no target action, a weighted push instead. There is no "correct" valve setting to learn from, so the actor has no labels in the supervised sense. Each training sample instead carries three numbers saved during the rollout: the action actually taken, a_t (the mean plus exploration noise); its log-probability under the policy that collected the data, $\log \pi_{\theta_{old}}(a_t \mid o_t)$; and its advantage $\hat{A}_t$. The actor minimizes $-L^{\text{clip}}$ of Eq. (11). In effect: if $\hat{A}_t > 0$, the action turned out better than the critic expected, and the actor's mean moves toward a_t. If $\hat{A}_t < 0$, it moves away. The change per round is capped at about $\pm 20\%$ in action probability. The exploration spread σ is a separate learned parameter, not a network output, and is adjusted by the same loss.

Where the data live. All of this is held in memory for one update only (`train_ppo.py`):

quantity	array	shape	used by
observations o_t	`obs_buf`	(T=256, N=16, obs dim)	actor and critic (input)
actions a_t	`act_buf`	(256, 16, action dim)	actor
$\log \pi_{\theta_{old}}(a_t \mid o_t)$	`logp_buf`	(256, 16)	actor (ratio ρ_t)
$V_{\phi_{old}}(o_t)$	`val_buf`	(256, 16)	GAE, critic label
rewards r_t	`rew_buf`	(256, 16)	GAE
episode-end flags d_t	`done_buf`	(256, 16)	GAE
advantages $\hat{A}_t$	`adv`	(256, 16)	actor (weight)
critic labels $\hat{R}_t$	`ret`	(256, 16)	critic (target)

Each update, the 16 workers return 256 steps each. The main process computes $\hat{A}_t$ and $\hat{R}_t$ in one backward pass, flattens the arrays into 4,096 samples, trains for 10 passes in minibatches of 256, and then overwrites everything with the next rollout. Only the final network weights are saved to disk (`results/ppo_*.pt`).

What about the poor early critic? At the start the critic's predictions are poor, so the early labels $\hat{R}_t$ (partly built from those predictions) are biased, and the early advantages are noisy. This does not linger: PPO is *on-policy*, so each batch of data is used for one round of training and then discarded. Nothing collected with the early policy and critic is reused later. Each new batch is collected with the improved actor and labelled with the improved critic, so the bias shrinks as training proceeds. The $\lambda = 0.95$ weighting, which leans mostly on actual rewards, and the clipped policy update limit the damage meanwhile. (This is unlike "off-policy" methods such as DQN or SAC, which keep a large replay memory of old experience.)

The exception: behavior cloning. In the warm start used for the pond (Section 6, last subsection), both networks do have ordinary supervised targets. The inputs are the observations from runs of the tuned rule. The actor's target is the rule's action (mean squared error on the actor's mean output). The critic's target is the plain discounted sum of the rewards the rule actually earned, with no bootstrapping. This is done once, before PPO starts, and it means the critic begins with sensible predictions instead of random ones.

6.6 The full training loop

PPO as implemented in `train_ppo.py`

1. Initialize actor θ, critic ϕ, $\log \sigma = -0.5$. Start 16 simulators, each with its own random season.
2. Repeat for 100 updates (single reservoir) or 245 updates (cascade):
   (a) *Rollout:* each simulator runs 256 days with the current stochastic policy, so 4,096 (o, a, r) records per update. A finished season immediately restarts with a new random season.
   (b) *Advantages:* compute δ_t, $\hat{A}_t$ (Eq. 10) and $\hat{R}_t$ for every record; standardize $\hat{A}_t$.
   (c) *Optimize:* 10 passes (epochs) over the 4,096 records in shuffled minibatches of 256; on each minibatch take one Adam gradient step on $L(\theta, \phi)$ (Eq. 12), with gradient norm clipped at 0.5.
3. Save θ (the trained policy).

6.7 Inside PyTorch: how the actor is built and how its weights change

This subsection opens the box: what the actor is in code, what PyTorch computes when it is trained, and *when* its weights change as new simulation results arrive.

The network is a function with adjustable coefficients. In `train_ppo.py` the actor is three linear layers with tanh between them:

```
self.actor = nn.Sequential(
    nn.Linear(obs_dim, 64), nn.Tanh(), # W1: 64 x obs_dim, b1: 64
    nn.Linear(64, 64), nn.Tanh(), # W2: 64 x 64, b2: 64
    nn.Linear(64, act_dim)) # W3: act_dim x 64, b3: act_dim
self.log_std = nn.Parameter(torch.full((act_dim,), -0.5))
```

Written out, the actor is the formula

$$\mu_\theta(o) = W_3 \tanh\big(W_2 \tanh(W_1 o + b_1) + b_2\big) + b_3, \tag{13}$$

and θ is the list of all entries of $W_1, b_1, W_2, b_2, W_3, b_3$ plus $\log\sigma$. For the single reservoir (o has 7 entries, one action) that is $7 \cdot 64 + 64 + 64 \cdot 64 + 64 + 64 + 1 = 4737$ coefficients plus $\log\sigma$. For the cascade (16 observations, 2 actions) it is 5378 plus two $\log\sigma$. The critic is a separate network of the same shape ending in one output, $V_\phi(o)$. `nn.Linear` starts each weight as a small random number (uniform in $\pm 1/\sqrt{\text{inputs}}$). The untrained actor is therefore a random, nearly flat function of the state, and its releases are near the middle of the range.

It helps to think of Eq. (13) as an ordinary nonlinear regression model with 4737 parameters. Training it is a calibration, but with an unusual objective. We never have a "measured" correct action to fit to. The objective is built from the simulated rewards instead.

From the network to a probability. `model.dist(obs)` returns the Gaussian $\pi_\theta(a \mid o) = \mathcal{N}(\mu_\theta(o), \sigma^2)$, and `log_prob` evaluates, for a recorded action a,

$$\log \pi_\theta(a \mid o) = -\frac{(a - \mu_\theta(o))^2}{2\sigma^2} - \log\sigma - \tfrac{1}{2}\log 2\pi \tag{14}$$

(summed over the action entries when there are two). Everything PPO does to the actor goes through this one number.

With one action (the single reservoir) this is a univariate Gaussian. With two actions (the cascade, or a release plus a valve setting) it is a bivariate Gaussian with a *diagonal* covariance, $\text{diag}(\sigma_1^2, \sigma_2^2)$: one mean and one spread per action and no correlation term, which is the same as two independent Gaussians. In code, `torch.distributions.Normal(mu, log_std.exp())` acts element by element and `log_prob(a).sum(-1)` adds the per-action terms of Eq. (14). The two *means* are still coupled, because both come out of the same network from the same observation. Only the exploration noise added to them is independent.

Where the spread σ comes from. σ is not computed from data. It is one more trainable parameter, updated by the same gradient steps as the weights. The network stores $\log\sigma$, one entry per action (`self.log_std = nn.Parameter(torch.full((act_dim,), -0.5))`), and uses $\sigma = e^{\log\sigma}$. Storing the logarithm keeps σ positive whatever value the optimizer gives it, and makes its steps relative: a step of 0.1 in $\log\sigma$ changes σ by about 10%. σ does not depend on the state. Exploration is equally wide whether the reservoir is full or empty. A state-dependent or correlated spread is possible, but this simpler form is the usual PPO default and was enough here.

Starting value. Plain PPO starts at $\log\sigma = -0.5$, i.e. $\sigma \approx 0.61$ on the action range $[-1, 1]$: wide exploration. After behavior cloning (Section 6.9) the start is set by hand (`--bc-log-std`). Cloning is a least-squares fit of μ_θ and carries no information about σ, and a policy that already imitates a good rule should explore only around it. The pond used $\log\sigma = -1.2$ ($\sigma \approx 0.30$).

How it is learned. Because $\log \sigma$ appears in Eq. (14), `loss.backward()` gives it a gradient like any weight, and Adam updates it. For one action entry,

$$\frac{\partial \log \pi_\theta(a \mid o)}{\partial \log \sigma} = \frac{(a - \mu_\theta(o))^2}{\sigma^2} - 1, \tag{15}$$

which is positive for an action farther than one σ from the mean and negative for one closer. The loss weights each record by its advantage $\hat{A}_t$. If the actions that turned out better than expected ($\hat{A} > 0$) lie far from μ, σ grows ("exploring wider paid off"). If they lie close to μ, or the far-out actions were the bad ones ($\hat{A} < 0$), σ shrinks ("stay near the mean"). σ therefore settles where the advantage-weighted spread of actions says further exploration no longer pays. We use no entropy bonus (`--ent-coef 0`), so nothing else keeps σ large, and in practice it shrinks as the policy improves. In the cascade run, the `std` column of the training log ($e^{\log \sigma}$) fell from 0.595 to 0.184 over 245 updates.

At evaluation σ is not used at all: the action is the mean, $a = \mu_\theta(o)$.

One training step, line by line. The update loop in `train_ppo.py` is:

```
d = model.dist(b_obs[idx]) # forward pass: mu, sigma for 256 records
logp = d.log_prob(b_act[idx]).sum(-1) # Eq. (logp) for the recorded actions
ratio = (logp - b_logp[idx]).exp() # rho = pi_theta / pi_old
pg_loss = -torch.min(ratio * b_adv[idx],
                     ratio.clamp(0.8, 1.2) * b_adv[idx]).mean() # -L_clip
v_loss = 0.5 * (model.value(b_obs[idx]) - b_ret[idx]).pow(2).mean()
loss = pg_loss + 0.5 * v_loss
opt.zero_grad() # forget the previous gradient
loss.backward() # dloss/dparameter for every parameter
nn.utils.clip_grad_norm_(model.parameters(), 0.5)
opt.step() # Adam changes the parameters
```

1. *Forward pass.* The 256 observations of the minibatch go through Eq. (13) at once (as a $256 \times$ obs_dim matrix). PyTorch records every operation as it computes, building a *computation graph* from the parameters to `loss`.

2. *Inputs that are just data.* `b_obs`, `b_act`, `b_logp` (the log-probability under the policy that collected the data, $\pi_{\theta_{old}}$), `b_adv` ($\hat{A}_t$) and `b_ret` ($\hat{R}_t$) were computed before the update and are constants here. In particular the advantages come from the critic *as it was* when the rollout ended. They are not recomputed while the actor changes.

3. *Backward pass.* `loss.backward()` applies the chain rule through the recorded graph, from `loss` back to every parameter, and stores $\partial L / \partial \theta_i$ in each parameter's `.grad`. This is *back-propagation*. It is the same thing as an adjoint sensitivity calculation, and it gives all 4737 derivatives for about the cost of one extra forward pass, instead of 4737 finite-difference runs.

4. *Gradient clipping.* If the length of the whole gradient vector exceeds 0.5 it is scaled down to 0.5. Its direction is kept and its size is capped, a guard against one unusual minibatch throwing the weights far.

5. *Adam step.* `opt.step()` changes each parameter (next paragraph). `zero_grad()` is needed because PyTorch *adds* new gradients to `.grad` rather than overwriting them.

What the gradient says, for one record. Take one record with advantage $\hat{A}$ and suppose clipping is not active. Its contribution to the actor loss is $-\rho\hat{A}$, and with Eq. (14) the chain rule gives

$$\frac{\partial(-\rho\hat{A})}{\partial\theta} = -\rho\,\hat{A}\,\frac{a-\mu_\theta(o)}{\sigma^2}\,\frac{\partial\mu_\theta(o)}{\partial\theta}. \tag{16}$$

Gradient descent moves θ *against* this. If $\hat{A} > 0$ (the recorded action turned out better than the critic expected), the step changes the weights so that $\mu_\theta(o)$ moves *toward* the action a that was actually taken. If $\hat{A} < 0$, it moves away from it. The factor $\partial\mu_\theta/\partial\theta$ is what back-propagation supplies. It says how each weight would have to change to move the network's output at this particular state o. Because the weights are shared by all states, the change also shifts the release at *similar* states. This generalization is what lets a few thousand coefficients represent a rule over a continuous state space. Once ρ leaves $[0.8, 1.2]$ in the direction the advantage favors, the clipped term is the active one, its gradient is zero, and that record stops pulling. The minibatch loss averages Eq. (16) over 256 records, so the step follows the consensus of many experiences, not any single one. The spread $\log\sigma$ gets its own gradient from the same loss. It shrinks when the advantage-weighted actions lie close to μ, which is how exploration narrows by itself as training proceeds.

The Adam update. Adam keeps two running averages for every parameter θ_i: of its gradient g_i and of g_i^2. With $\beta_1 = 0.9$, $\beta_2 = 0.999$, step count k and learning rate $\alpha = 3 \times 10^{-4}$:

$$\begin{aligned} m_i &\leftarrow \beta_1 m_i + (1-\beta_1) g_i, & v_i &\leftarrow \beta_2 v_i + (1-\beta_2) g_i^2, \\ \theta_i &\leftarrow \theta_i - \alpha\,\frac{m_i/(1-\beta_1^k)}{\sqrt{v_i/(1-\beta_2^k)} + 10^{-8}}. \end{aligned} \tag{17}$$

m_i is a smoothed gradient (momentum) and $\sqrt{v_i}$ its typical size, so each parameter moves by roughly α per step in the direction its recent gradients agree on, whatever its scale. A weight whose gradient keeps flipping sign moves little. The divisions by $1-\beta^k$ correct the start-up bias of the averages, which begin at zero. One optimizer (one set of m, v) serves actor, critic and $\log\sigma$ together, and it persists across updates, so momentum carries over from one batch of data to the next.

When the weights change as new data arrive. The weights are *not* updated after every simulated day. Training alternates between two phases:

1. *Collect, weights frozen.* The current weights are copied to the 16 simulator workers. Each runs 256 days with them, drawing random actions from $\mathcal{N}(\mu_\theta(o), \sigma^2)$ and recording o, a, $\log\pi_{\theta_{old}}(a \mid o)$, $V_\phi(o)$ and r. Nothing learns during this phase. The 4,096 new records are then sent back.
2. *Learn, data frozen.* Advantages and returns are computed for the 4,096 records (Section 6.5). The records are then reused for 10 epochs. Each epoch shuffles them into 16 minibatches of 256 and takes one Adam step per minibatch, so there are 160 weight updates per batch of data. In the first minibatch $\theta = \theta_{old}$ and every $\rho = 1$. As the steps accumulate, ρ drifts away from 1 and the clipping starts to switch records off.

After the 160 steps the records are *discarded*, because they were generated by a policy that no longer exists, and the new weights go back to the workers for the next batch. Over the single-reservoir

run this cycle repeats 100 times: 409,600 simulated days and 16,000 Adam steps. Each new batch of experience therefore enters the actor in three ways: as the actions a (where to move μ), as the advantages $\hat{A}$ (which way and how strongly), and, through the critic that is fitted on the same records, as better advantages in the next batch.

Two related procedures use the same machinery. *Behavior cloning* (Section 6.9) trains the same actor with the same Adam optimizer, but its loss is ordinary least squares, $\frac{1}{n}\sum(\mu_\theta(o) - a^{\text{rule}})^2$, with the rule's actions as targets. That is regression in the plainest sense. At *evaluation* no gradients are computed (`torch.no_grad()`): the trained actor is just Eq. (13) evaluated with fixed coefficients, and its output is the release decision.

6.8 Hyperparameters and what they control

setting	value	what it controls
parallel simulators	16	throughput; one OHQ instance per CPU worker
rollout length	256 days	experience collected per simulator per update
samples per update	4,096	size of the batch the policy learns from each round
epochs per update	10	how many times each batch is reused
minibatch size	256	records per gradient step
learning rate (Adam)	3×10^{-4}	gradient step size
discount γ	0.99	how far ahead the agent looks (effective horizon $\sim 1/(1-\gamma) = 100$ days)
GAE λ	0.95	bias–variance trade-off of the advantage estimate
clip ϵ	0.2	maximum change in action probabilities per round (trust region)
value coefficient c_v	0.5	weight of the critic's loss
gradient-norm clip	0.5	guards against occasional huge gradients
total steps	409,600 / $\sim 10^6$	training budget, single reservoir / cascade

These are standard PPO defaults. None were tuned for this problem.

Parallel rollouts. The 16 simulators run in separate worker processes. Each owns its own OHQ instance and working folder. At every update the main process sends the current network weights to all workers. Each worker then collects its 256 steps independently and returns them; when a season ends it starts a new one immediately. Earlier versions used Gymnasium's vectorized environment, which advances all simulators in lockstep. That was fine while every step took the same few milliseconds, but in the stormwater problem rare slow steps (about one hour in 150 takes about a second) made every step wait for the slowest simulator, cutting throughput by an order of magnitude.

A performance pitfall. The first parallel run made almost no progress. Every worker sat at $\approx 370\%$ CPU because OHQ's solver defaults to `n_threads = 8` (OpenMP), and BLAS/PyTorch add their own threads. Sixteen workers times eight threads oversubscribed the 32-core machine. Whenever OHQ models run in parallel, `n_threads` must be 1. With `n_threads = 1` in OHQ and `OMP_NUM_THREADS = 1` / `torch.set_num_threads(1)` in the workers, throughput rose to $\approx$ 1,200 environment steps per second.

6.9 Two optional additions used for the stormwater pond

Reward normalization. When most steps cost almost nothing and a few cost thousands of times more (an hour of pond overflow), the critic's least-squares fit is dominated by the rare spikes and the advantage estimates become noisy. With `--normalize-rewards`, every reward is divided by a running estimate of the standard deviation of the discounted return and clipped to $[-10, 10]$. Rescaling all rewards by a constant does not change which policy is best. Clipping, however, changes the objective by capping the influence of the most extreme hours. In the pond those hours (overflows) are exactly what drives the exceedance flow, so the final pond run scales without clipping (`--reward-clip 0`).

Choosing the policy by the true objective. The reward PPO optimizes is only a stand-in for the evaluation objective, so the last policy of a run is not necessarily the best by that objective. With `--save-every`, the trainer saves snapshots of the networks during training, including the starting (cloned) policy. Afterwards, `select_checkpoint.py` runs each snapshot on validation periods, computes the true objective, and keeps the best. This is the RL counterpart of early stopping on a validation set in ordinary model fitting.

Starting from a rule: behavior cloning. PPO normally starts from a random policy and must discover everything by trial and error. With `--bc-rule`, each worker first runs a given rule-based policy (here the tuned forecast rule) for a few thousand steps. The actor is then fitted by ordinary least squares to reproduce the rule's actions, and the critic to the returns the rule achieved. PPO then starts from a policy that already behaves like the rule, with a narrower exploration spread ($\sigma = 0.3$), and can only improve on it if the data support doing so. This is a common and practical way to combine engineering knowledge with RL. The rule encodes what operators already know, and RL looks for improvements around it.

7 Results for the single reservoir

7.1 How we judge a policy

A learned policy is only useful if it performs well on seasons it has never seen. The procedure, used throughout, is:

- **Held-out test seasons.** Thirty seasons generated from seeds that were never used in training (seeds 10,000–10,029). Every policy faces exactly the same 30 storm sequences.
- **Score.** The season return of Eq. (8): minus the total penalty, so 0 is perfect and higher is better. We also report its components (supply, flood, spill, terminal).
- **Paired comparison.** Some seasons are much stormier than others, so the spread of returns across seasons is large for *every* policy. Comparing means alone mixes up policy quality with weather luck. Because all policies see the same seasons, we instead compute, season by season, the difference in return between PPO and a baseline. We report its mean and its standard error (standard deviation over $\sqrt{30}$). A mean difference larger than about twice its standard error is unlikely to be luck. We also count in how many of the 30 seasons PPO did better.
- **Fair baselines.** A comparison is only as good as its opponent, so the rule curve's parameter is tuned on a separate set of seasons before testing.

7.2 Training

Training used 409,600 environment steps (100 PPO updates, about 4,500 episodes), which took about 5 minutes with 16 workers at $\approx$ 1,450 steps/s. Each step is one complete OHQ model build and solve, so this is also about 4×10^5 OHQ runs. Figure 7 shows that the mean return rises from about -52 (a near-random policy) to about -20 within the first $\sim$150k steps and then plateaus. The exploration standard deviation keeps shrinking (0.59 $\rightarrow$ 0.15). The remaining noise in the curve comes from the storm-to-storm variability of the episodes, not from the policy: a season with five large storms costs far more under *any* policy than a dry one.

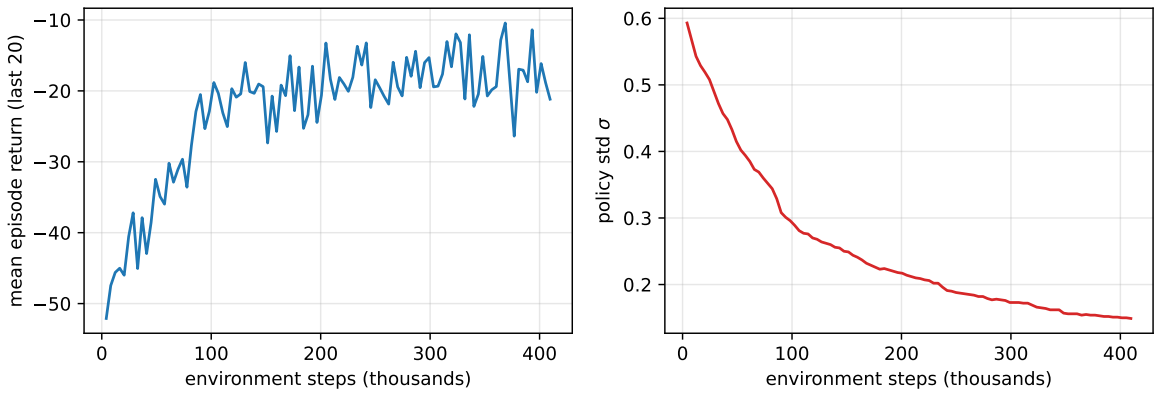


Figure 7: Left: mean return of the last 20 completed training episodes. Right: the policy's exploration standard deviation.

7.3 Comparison with baselines

We evaluate three policies on the same 30 held-out storm sequences (seeds 10,000–10,029, never seen in training):

Demand only: release $Q_t = D_t$.

Rule curve (0.8): a standard operating policy with a flood rule: release D_t, and above ηS_{cap} also release the excess storage over one day: $Q_t = \min\left(Q_{\text{safe}},\, D_t + (S_t - \eta S_{\text{cap}})/\Delta t\right)$ with $\Delta t = 1$ d, and $\eta = 0.8$.

Tuned rule: the same rule with η tuned by grid search on separate seeds (see below).

PPO: the trained agent (deterministic).

Tuning the baseline. The 0.8 threshold of the rule curve was an arbitrary first choice. To give the agent a fair opponent, we also grid-searched the threshold on 30 separate *tuning* seeds (20,000–20,029; Figure 8). The best value, a flood rule that starts releasing above 0.4 S_{cap}, is then evaluated on the same test seeds as everything else ("Tuned rule").

Paired comparison. Because every policy sees the same 30 storm sequences, the fair test is the per-episode difference in return (Table 3). The spread in Table 2 is dominated by how stormy each season is, and this pairing cancels it.

Policy	Return	Supply	Flood	Spill	Terminal	Deficit [%]	Flood days
Demand only	-38.41 ± 35.47	-1.24	-20.39	-16.33	-0.43	2.2	9.3
Rule curve (0.8)	-22.56 ± 26.30	-1.31	-13.76	-6.88	-0.61	2.3	6.9
Tuned rule (0.4)	-15.81 ± 23.95	-2.95	-7.73	-3.87	-1.25	5.6	3.5
PPO	-15.01 ± 23.38	-2.88	-7.50	-3.39	-1.24	6.0	6.6

Table 2: Mean over 30 test episodes. The return is split into its supply, flood, spill and terminal components (all penalties, so higher is better). *Deficit* is the fraction of total demand not delivered. *Flood days* counts days with river flow above Q_{safe}.

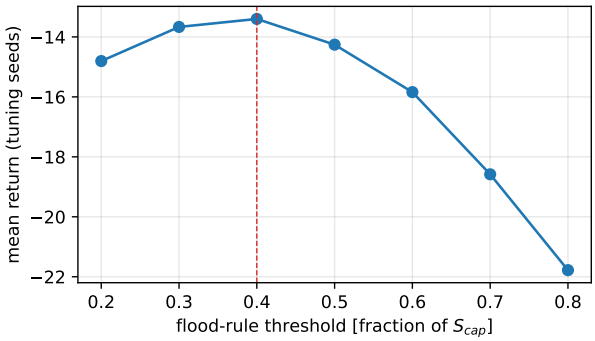


Figure 8: Mean return of the rule curve versus its flood-rule threshold, on the tuning seeds.

What the agent learned.

- PPO clearly beats releasing the demand only (by about 23 return points) and the untuned rule curve (by about 7.5 points, better in 20 of 30 episodes). It cuts the flood penalty roughly in half and the spill penalty by a factor of about two.
- It does this with a *drawdown hedging* strategy (Figure 10). PPO releases more than the demand most of the time and keeps the reservoir low, so storms are absorbed instead of spilled. The cost is a higher supply deficit (6% versus 2% of demand) and occasional near-empty periods. The policy map (Figure 12) shows the release increasing smoothly with both storage and the forecast inflow.
- Against the *tuned* rule curve the difference is $+0.8 \pm 0.9$, i.e. statistically indistinguishable. In effect, PPO rediscovered "keep the reservoir around 40% full and release the excess", which is close to the best policy in the rule-curve family for this reward. The two policies reach the same return differently: the tuned rule floods on fewer days, while PPO exceeds Q_{safe} more often but by smaller margins.

Interpretation. For a single reservoir with a 3-day forecast and this reward, a well-tuned one-parameter rule is already near-optimal. RL earns its keep only when that stops being true, for example with multiple interacting reservoirs, nonlinear water-quality objectives, longer forecasts whose skill varies, or rewards that a fixed rule cannot express. The value of this exercise is the pipeline: the same environment-and-agent code applies unchanged to richer OHQ models.

baseline	mean(PPO − baseline)	std. error	PPO better in
Demand only	+23.39	4.01	23/30
Rule curve (0.8)	+7.54	2.11	20/30
Tuned rule (0.4)	+0.79	0.86	12/30

Table 3: Paired differences in episode return, PPO minus baseline, on the 30 test episodes.

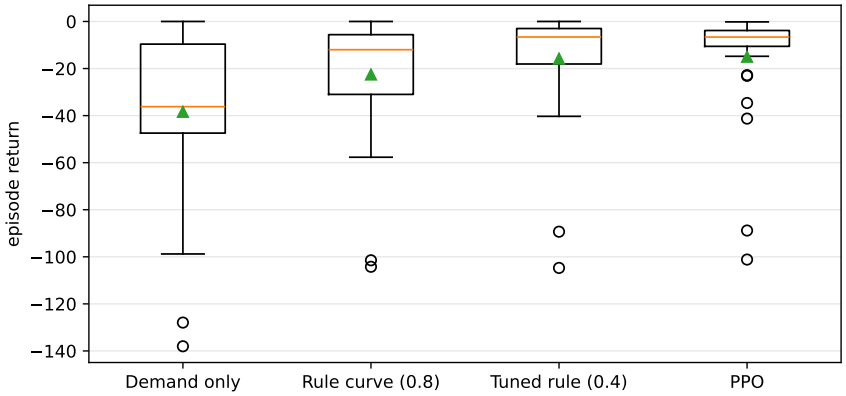


Figure 9: Distribution of test-episode returns.

8 Performance and the case for a codegen backend

RL needs a very large number of short simulations; this run alone used about 4×10^5 one-day OHQ solves. Figure 13 breaks down the cost of one step.

One step costs about 4.3 ms, but the numerical solve (20 implicit steps of 0.05 d) accounts for only 0.9 ms. The remaining 3.5 ms (80%) is overhead: constructing a `System` and parsing the JSON templates (2.2 ms), building blocks and links (0.9 ms), and writing and parsing the output file (0.4 ms). The overhead is a fixed per-step cost, independent of how hard the physics is. A codegen-compiled model that is built once and only has its state reset between steps would remove nearly all of it, giving roughly a 5× speed-up for this model even before any gain in the solver itself.

Backend contract. Any replacement only needs to implement

```
class Backend:
    def set_inflow(self, t, q): ... # forcing for the whole episode
    def simulate_day(self, t0, S0, Q): # one control interval
        return S1, released_volume, spilled_volume
    def close(self): ...
```

and is passed to the environment as `OHQReservoirEnv(backend=...)`. A compiled kernel that keeps its state in memory between calls would also remove the per-step file round trip and the template parsing, and would allow batched, vectorized rollouts within a single process.

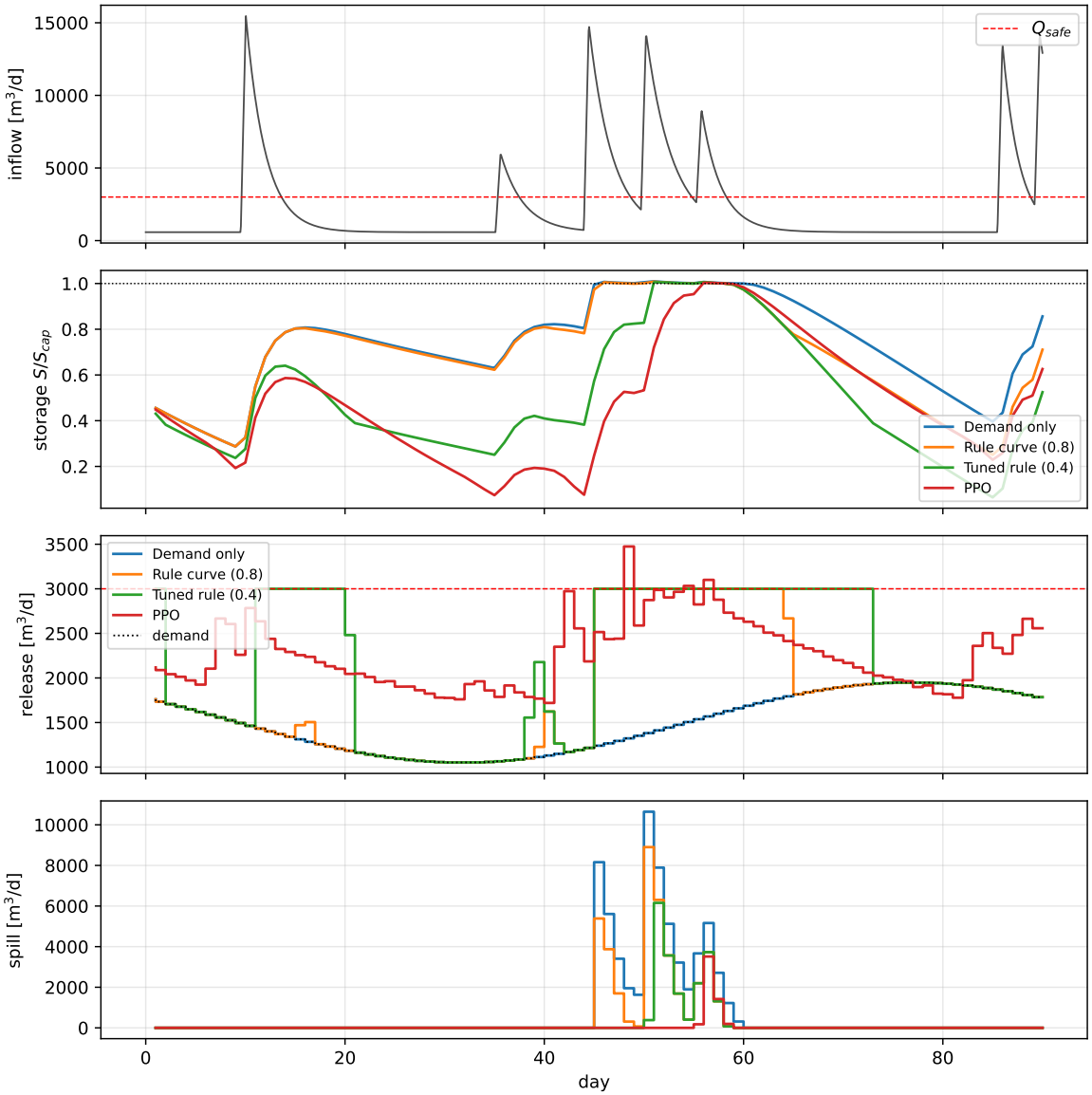


Figure 10: Test episode with the largest PPO gain over the rule curve (0.8). From top: inflow, storage, release (dotted: demand; dashed: Q_{safe}), spill.

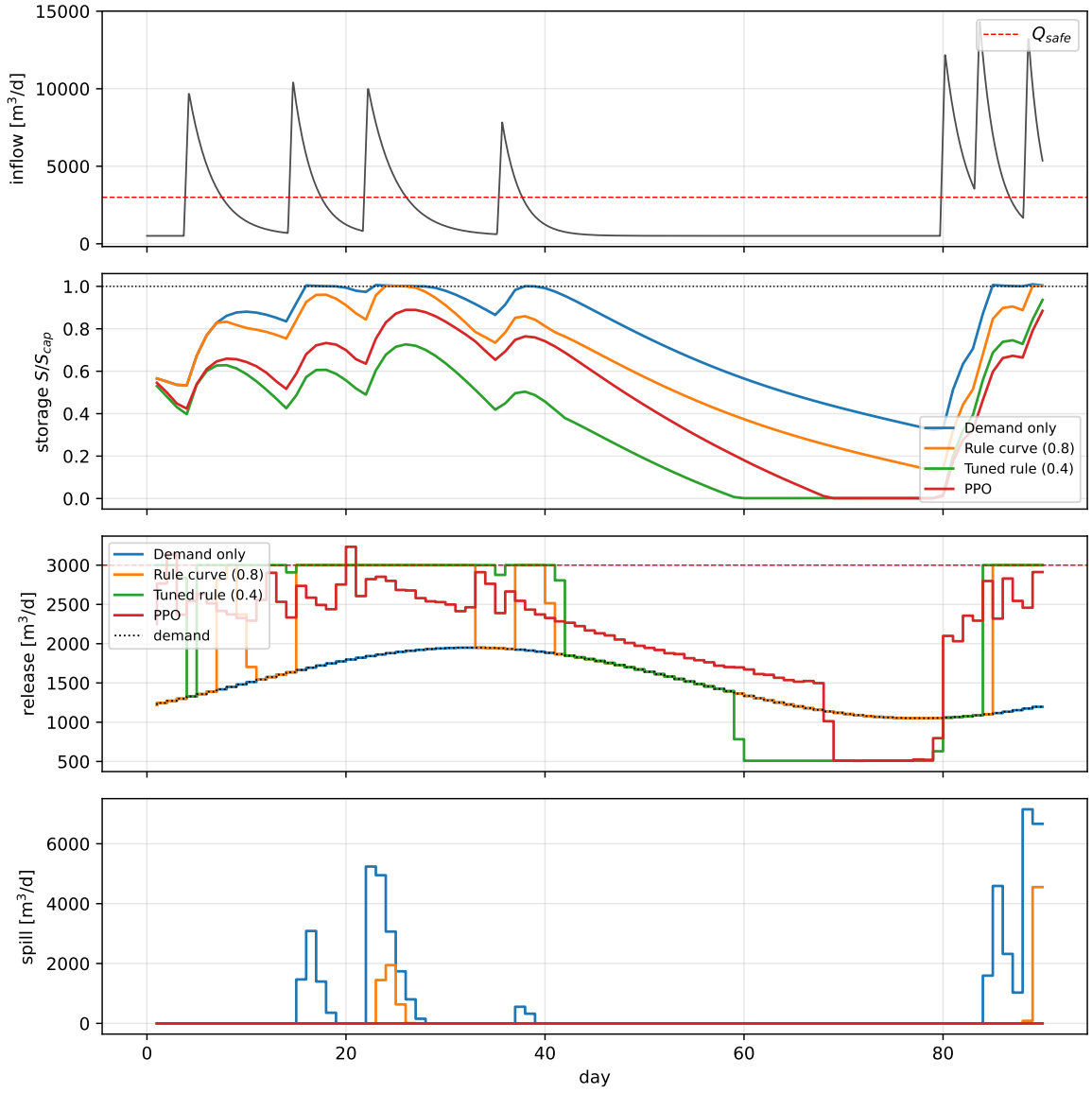


Figure 11: A test episode with the median PPO-vs-rule-curve (0.8) difference.

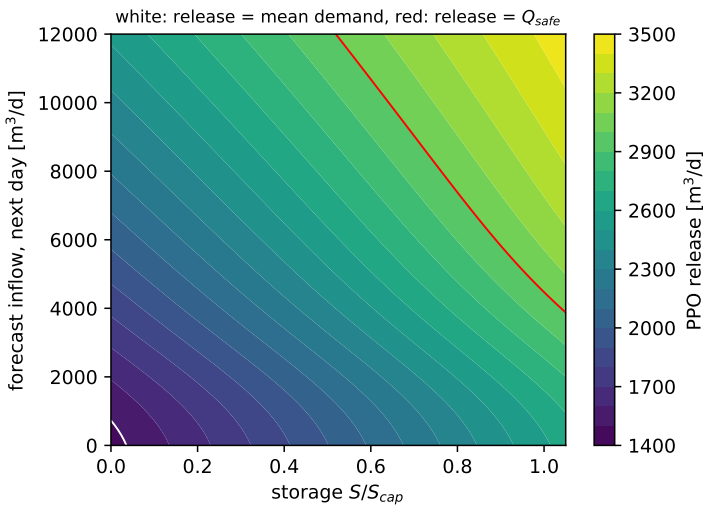


Figure 12: Learned release as a function of storage and the inflow forecast. The current inflow and the 1- and 2-day forecasts are set to the value on the vertical axis, the 3-day forecast to 0.6× that value, with mean demand at mid-season.

9 A harder problem – a two-reservoir cascade

The single reservoir turned out to be easy enough that a tuned one-parameter rule matched PPO. Here the agent operates two reservoirs in series. An uncontrolled tributary joins below them, so the agent must coordinate where to hold flood space and when to release.

9.1 System

Storms are regional: every storm hits all three catchments, with each catchment responding at its own scale and speed (all flows in m^3/d):

catchment	base flow	peak scale $\bar{P}$	delay	rise time	recession k [d^{-1}]
A (upstream, large)	500	5,000	0.5 d	1.0 d	$U(0.2, 0.4)$
B (local)	200	3,000	0	0.5 d	$U(0.4, 0.7)$
tributary	300	4,000	0	0.25 d	$U(0.7, 1.2)$

Storm onsets arrive with probability 0.05 per day. Each storm has a common intensity $m \sim U(0.5, 1.5)$, and catchment c gets peak $\bar{P}_c\, m\, U(0.8, 1.2)$ with the hydrograph shape of Section 3.4. All base flows share one factor $\beta \sim U(0.7, 1.3)$. The demand at the flood point has mean $\bar{D} =$ 2,000 m^3/d with a ±30% seasonal swing. The river's safe capacity is $Q_{\text{safe}} =$ 4,000 m^3/d. The outlet capacities are $Q^A_{\text{out}} =$ 6,000 and $Q^B_{\text{out}} =$ 8,000 m^3/d.

Calibrating the storm climate. The first parameter set (storm probability 0.07, peak scales 7,000/4,000/6,000) was too wet. Mean inflow was about 314,000 m^3 per season, against about 280,000 m^3 the river can carry below Q_{safe} after the tributary. Even a rule that releases at the safe limit every day spilled more than 100,000 m^3, so penalties were dominated by unavoidable floods. With the values above, the mean inflow is about 200,000 m^3 (A+B) plus 53,000 m^3 (tributary). All three penalty types then matter, and the rule thresholds trade supply against flooding (Figure 16).

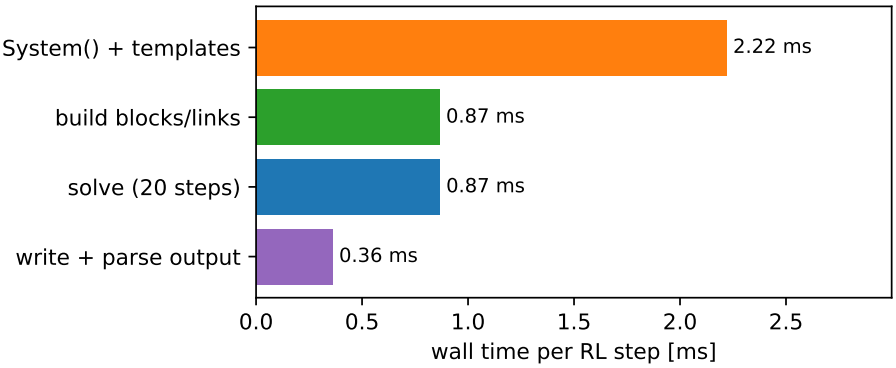


Figure 13: Wall time per environment step for the interpreted OHQ backend (mean of 50 steps, single thread).

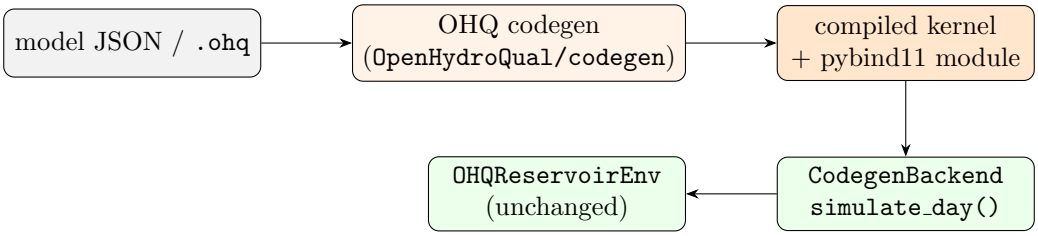


Figure 14: Planned replacement of the interpreted backend.

9.2 OHQ model

Per decision step, the backend (`CascadeBackend`) builds two `Reservoir` blocks and two `fixed_head` sinks (`River`, `SpillSinkB`), plus four `Reservoir_link_rule` links: `outletA` and `spillwayA` (A→B), `outletB` (B→`River`) and `spillwayB` (B→`SpillSinkB`). They use the same outlet and spillway parametrization as Section 4. B's released and spilled volumes come from the sink storages, as before. A's outflow enters B rather than a sink, so its volumes are recovered from A's storage at the end of every solver step. OHQ's time integration is backward Euler, so the volume through a link over a step of length Δt_0 is $Q(S_{\text{end of step}})\,\Delta t_0$:

$$V_t^{rel,A} = \sum_n Q_{\text{outlet}}\big(S_n^A\big)\,\Delta t_0, \qquad V_t^{sp,A} = \sum_n Q_{\text{spillway}}\big(S_n^A\big)\,\Delta t_0.$$

Check. B's mass balance, which includes these recovered A volumes as inflow, closes over a season to within 0.5 m^3 of ∼200,000 m^3 inflow. A's own balance closes to within 17 m^3. Daily residuals of ±100–200 m^3 on storm days are the same inflow-sampling effect as in Figure 6, and they cancel. One cascade step costs about 10 ms.

9.3 MDP and objective

Observation (16): S_t^A/S_{cap}^A, S_t^B/S_{cap}^B; the current inflows of A, B and the tributary; noisy 1-, 2- and 3-day forecasts of the daily-mean inflow of each of the three streams (9 values, same error model as before); D_t; and $1 - t/T$. Flows are divided by Q_{safe}.

Action (2): $a_t \in [-1, 1]^2$, mapped to $Q_t^A = \frac{1}{2}(a_{t,1} + 1)Q_{\text{out}}^A$ and $Q_t^B = \frac{1}{2}(a_{t,2} + 1)Q_{\text{out}}^B$.

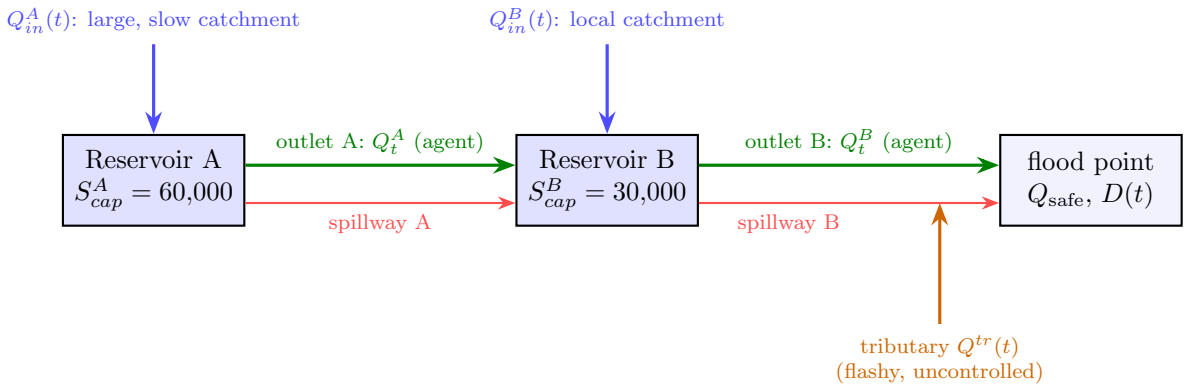


Figure 15: The two-reservoir cascade. Reservoir A's outflow, through its outlet or spillway, enters reservoir B. The flood point and the downstream users sit below the tributary confluence.

Reward: with river flow at the flood point $V^{riv}_t = V^{rel,B}_t + V^{sp,B}_t + V^{tr}_t$ (all over day t; V^{tr}_t is the tributary volume),

$$r_t = -\Big(\frac{\max(0, D_t - V^{riv}_t)}{D_t}\Big)^2 - w_f \frac{\max(0, V^{riv}_t - Q_{\text{safe}})}{Q_{\text{safe}}} - w_s \frac{V^{sp,A}_t + V^{sp,B}_t}{Q_{\text{safe}}},$$

plus the terminal term $-\frac{w_T}{2}\big[\max(0, \frac{1}{2}S^A_{cap} - S^A_T)/S^A_{cap} + \max(0, \frac{1}{2}S^B_{cap} - S^B_T)/S^B_{cap}\big]$, with the same weights as before.

The objective is again the expected undiscounted season return $J(\pi) = \mathbb{E}_\xi\big[\sum_t r_t + r^{\text{term}}_T\big]$, Eq. (8).

9.4 Baseline: a coordinated two-threshold rule

The rule has thresholds η_A, η_B (fractions of each capacity) and uses the current tributary flow q^{tr}_t and B's current inflow q^B_t (both m^3/d). With $\Delta t = 1$ d the drawdown time of excess storage, Write the storage above each threshold as a flow by spreading it over one day, and B's spare room per day:

$$E_A = \frac{S^A_t - \eta_A S^A_{cap}}{\Delta t}, \qquad E_B = \frac{S^B_t - \eta_B S^B_{cap}}{\Delta t}, \qquad R_B = \max(0, -E_B) + Q^B_t - q^B_t \quad (\text{all } \text{m}^3/\text{d}).$$

Then

$$\begin{aligned} Q^{sup}_t &= \max(0,\ D_t - q^{tr}_t), \\ Q^B_t &= \begin{cases} \max\big(Q^{sup}_t,\ \min(Q_{\text{safe}} - q^{tr}_t,\ Q^{sup}_t + E_B)\big) & E_B > 0 \\ Q^{sup}_t & E_B \le 0, \end{cases} \\ Q^A_t &= \mathbf{1}[E_B < 0]\ \max(0,\ Q^{sup}_t - q^B_t)\ +\ \mathbf{1}[E_A > 0]\ \min\big(E_A,\ \max(0, R_B)\big). \end{aligned}$$

In words: Q^{sup}_t is the part of the demand the tributary does not supply. B releases it, plus its excess storage as far as the river capacity left by the tributary allows. A keeps B supplied while B is below its target, and releases its own excess as far as B has room for it. both clipped to the outlet capacities. A first version backfilled *everything* B released, including flood releases, from A. That kept B full, and the rule spilled about 90,000 m^3 per season regardless of its thresholds. With the backfill restricted as above, the rule behaves sensibly. "Supply only" is this rule with $\eta_A = \eta_B = 1$ (never release for flood space). The tuned rule's thresholds are chosen by grid search over $\{0.1, \ldots, 0.9\}^2$ on 30 tuning seasons.

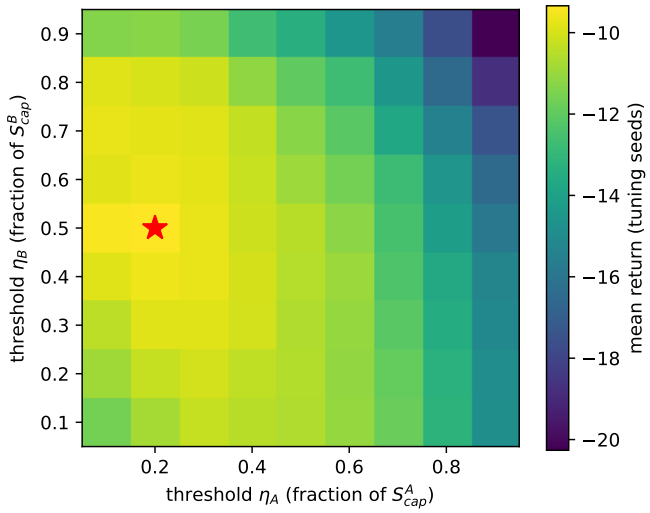


Figure 16: Mean return of the cascade rule over its two thresholds (tuning seasons). The star marks the tuned rule.

9.5 Training

PPO uses the same networks and hyperparameters as in Section 6, with 16 inputs and two action dimensions, for about 10^6 environment steps with 16 workers (about 670 steps/s). Training took about 28 minutes, about 10^6 simulated days or 11,000 seasons, each day a full OHQ run of the two-reservoir model. Figure 17 shows the return rising from about -44 (an untrained, essentially random policy) to about -11 after 160,000 steps, and to the -4 to -9 range after 500,000 steps. The curve stays jagged because it averages only the last 20 training seasons, and those seasons differ in storminess (and are run with exploration noise on). The exploration spread σ falls steadily from 0.59 to 0.18. The agent becomes more confident but still explores, so more training might still help a little.

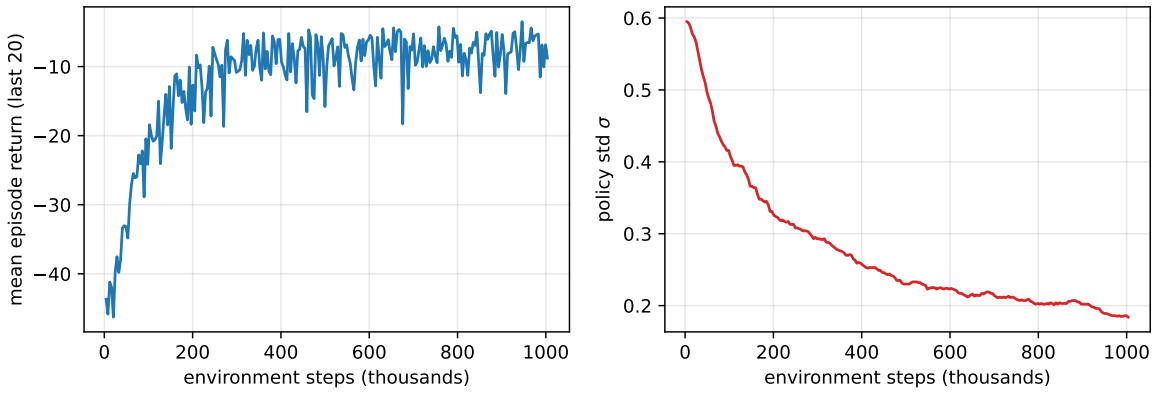


Figure 17: Cascade training: mean return of the last 20 episodes (left) and the mean policy standard deviation (right).

9.6 Results

All policies are evaluated on the same 30 held-out seasons (seeds 10,000–10,029). The tuned thresholds are $\eta_A = 0.2$, $\eta_B = 0.5$.

Policy	Return	Supply	Flood	Spill	Terminal	Deficit [%]	Flood days
Supply only	-38.59 ± 38.93	-1.12	-14.23	-22.74	-0.50	3.1	9.4
Rule (0.6, 0.5)	-15.94 ± 22.12	-1.18	-7.41	-6.64	-0.71	2.8	8.2
Tuned rule (0.2, 0.5)	-14.07 ± 19.26	-2.19	-6.30	-4.48	-1.10	5.1	11.4
PPO	-12.27 ± 17.30	-1.78	-6.01	-3.45	-1.03	5.3	8.5

Table 4: Cascade: mean over 30 test seasons, with columns as in Table 2.

baseline	mean(PPO − baseline)	std. error	PPO better in
Supply only	+26.32	5.47	21/30
Rule (0.6, 0.5)	+3.67	1.34	18/30
Tuned rule (0.2, 0.5)	+1.79	0.67	22/30

Table 5: Cascade: paired differences in season return, PPO minus baseline.

Headline. In the cascade, PPO beats every baseline, including the rule with tuned thresholds. Against the tuned rule, the season-by-season difference is $+1.79 \pm 0.67$: about 2.7 standard errors, with PPO better in 22 of the 30 test seasons (Table 5). In the single-reservoir problem the same comparison was a statistical tie. The extra difficulty of coordinating two reservoirs around an uncontrolled tributary is what lets the learned policy pull ahead. The size of the gain over the tuned rule is modest (about 13% of its penalty), but it is consistent.

Where the gain comes from. Table 4 splits the return into its parts. Compared with the tuned rule, PPO spills 23% less water (13,800 versus 17,900 m^3 per season), floods the river on fewer days (8.5 versus 11.4), and has a slightly smaller supply penalty with the same total shortfall ($\approx 5\%$ of demand). The three trajectory figures show the mechanisms:

- **Hedging in droughts (Figure 19).** In this median season a long dry spell empties both reservoirs under every policy. The rules deliver the full demand until the water runs out and then deliver almost nothing for about 20 days. PPO starts delivering somewhat less than demand around day 15, *before* the reservoirs are empty, and stretches its storage further. Because the supply penalty is *squared*, many small shortfalls cost less than a few large ones: spreading a 1,000 m^3/d deficit over twice as many days at half the size halves its penalty. This is the classic *hedging rule* of reservoir operation theory. The agent was never told about it; it discovered it from the shape of the reward.

- **Anticipating storms (Figure 20).** In this storm-rich season PPO raises releases ahead of forecast storms, up to and slightly above Q_{safe}, and keeps more room in B, so the storms around days 40–47 cause far less spill than under the rules.

- **Where it loses (Figure 21).** In the season where PPO does worst relative to the tuned rule, a cluster of storms arrives at the very end of the season. The tuned rule happened to

be holding A very low (its 20% threshold) and absorbed more of it. PPO, which also weighs the end-of-season storage term, was fuller and spilled more. A learned policy is better *on average*, not in every season.

The rule baselines. The threshold map (Figure 16) shows the logic of the best rule: keep the large upstream reservoir low ($\eta_A = 0.2$) and the small downstream one at half ($\eta_B = 0.5$), so that A's slow, large storm volumes have room while B buffers the fast local response. Even tuned, the rule cannot *change* these targets with the forecast or the season, and that flexibility is what PPO adds. "Supply only", which never releases for flood space, shows how much is at stake: it spills about 91,000 m^3 per season and scores −38.6.

Lessons for applying RL.

1. RL helps most where simple rules are structurally limited: several interacting storages, uncontrolled inflows and forecasts. For one reservoir, a well-tuned rule was as good.
2. The reward shapes the behavior. The hedging seen here follows from the squared deficit penalty, and a linear penalty would not produce it.
3. Fair comparison requires tuned baselines, identical test seasons and paired statistics. Against the untuned rule the gain would have looked twice as large (+3.7).

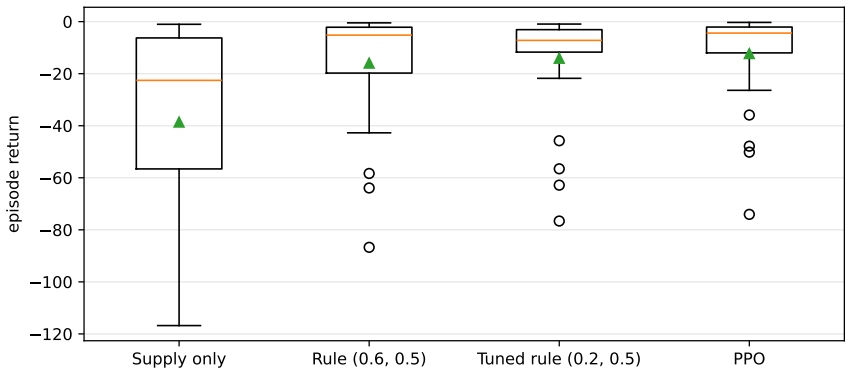


Figure 18: Cascade: distribution of test-season returns.

10 A third problem – real-time control of a stormwater pond

The third problem changes the time scale from days to hours and adds a water-quality objective. A stormwater pond collects runoff from an impervious catchment and releases it through a valve that can be opened or closed every hour. The pond has two jobs that pull in opposite directions:

- **Peak-flow control:** keep the outflow downstream low. We measure this with Q_p, the outflow exceeded only a fraction p of the time (the "exceedance" objective of OHQ, as in the St. Francis real-time-control study).
- **Treatment:** hold water long enough for pollutants to settle and degrade. We measure this with the *young-water fraction* of the outflow from OHQ's Laplace age tracer.

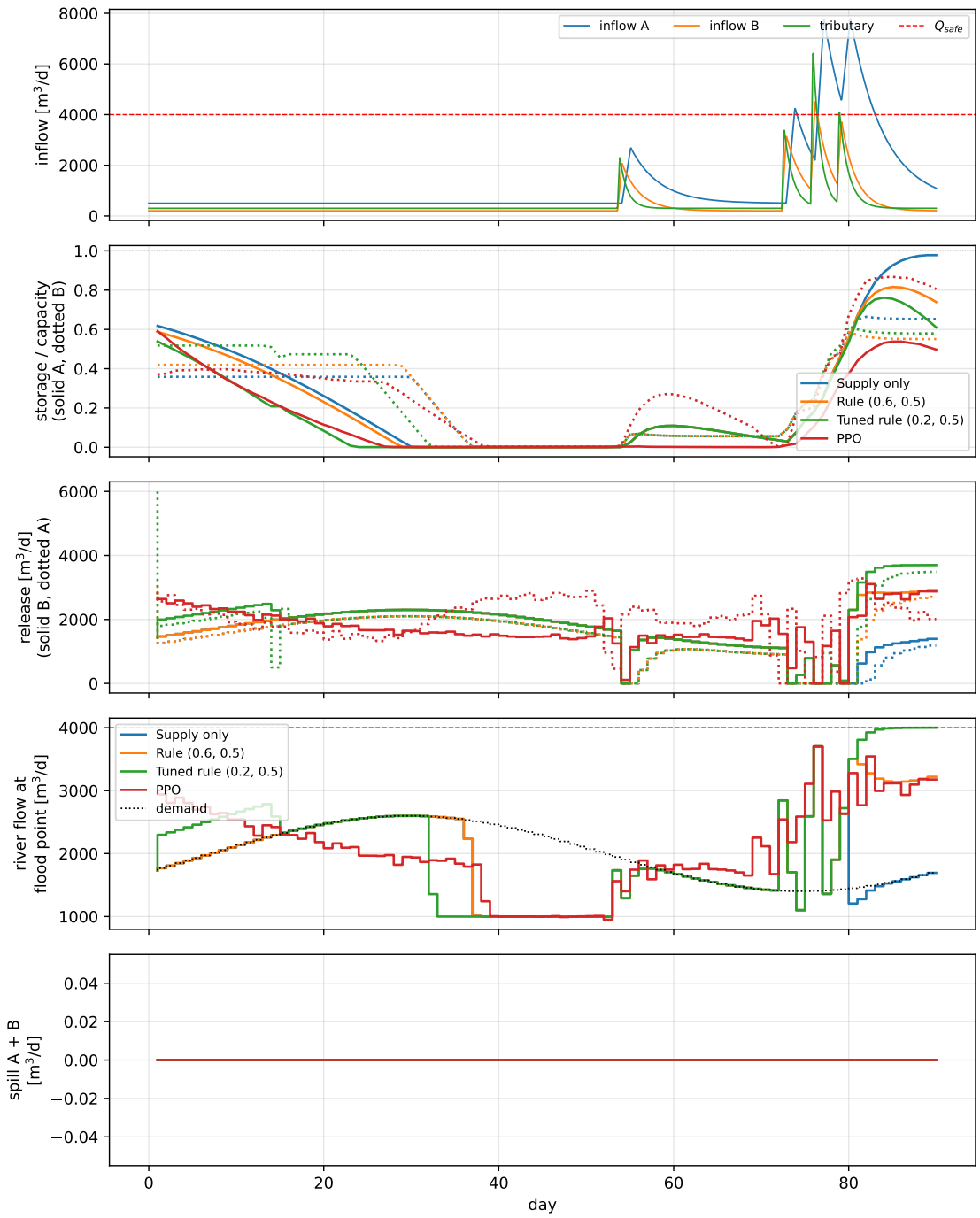


Figure 19: Cascade test season with the median PPO-vs-tuned-rule difference. From top: the three inflows; storages (solid A, dotted B); releases (solid B, dotted A); river flow at the flood point with demand (dotted) and Q_{safe} (dashed); total spill.

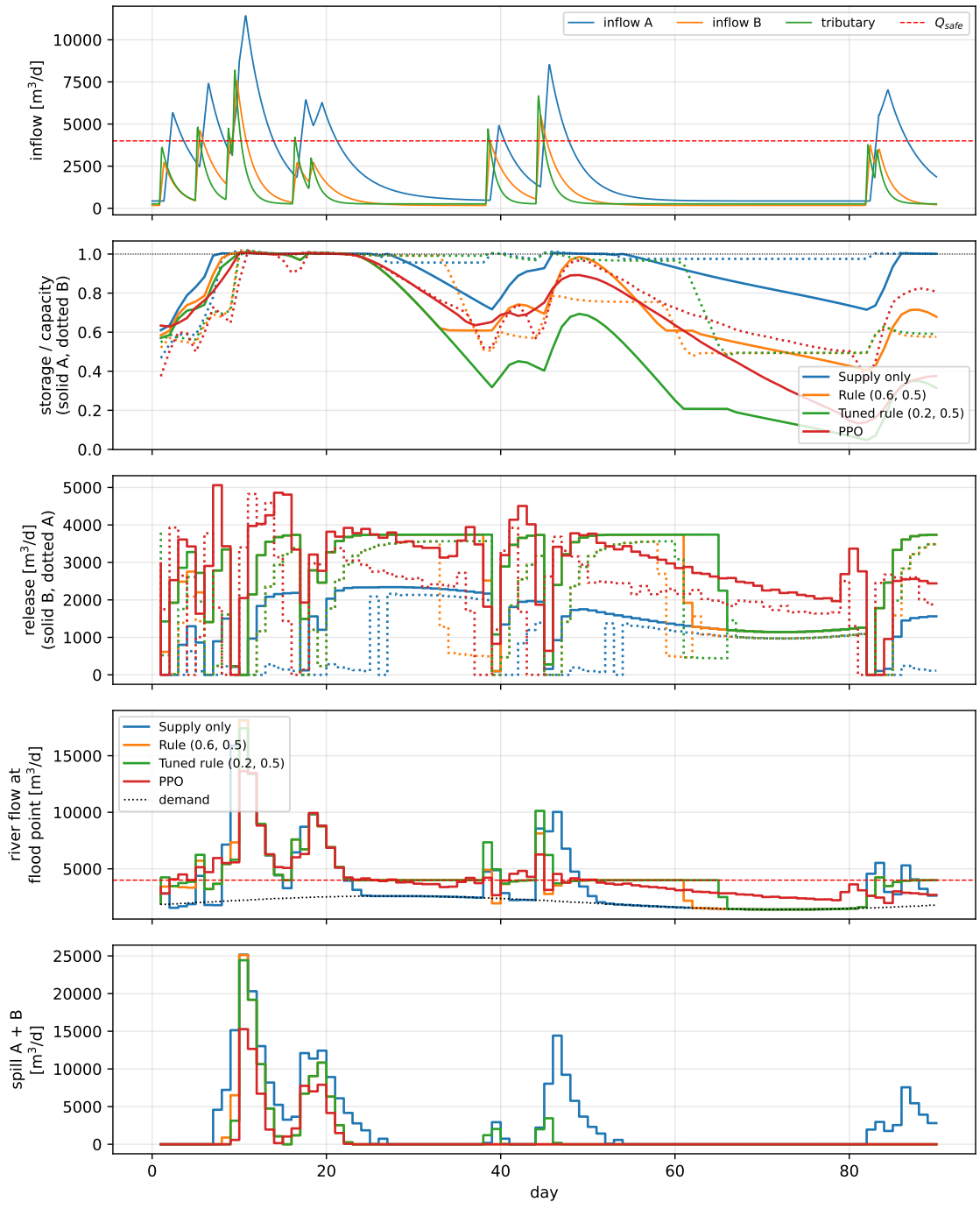


Figure 20: Cascade test season with the largest PPO gain over the tuned rule.

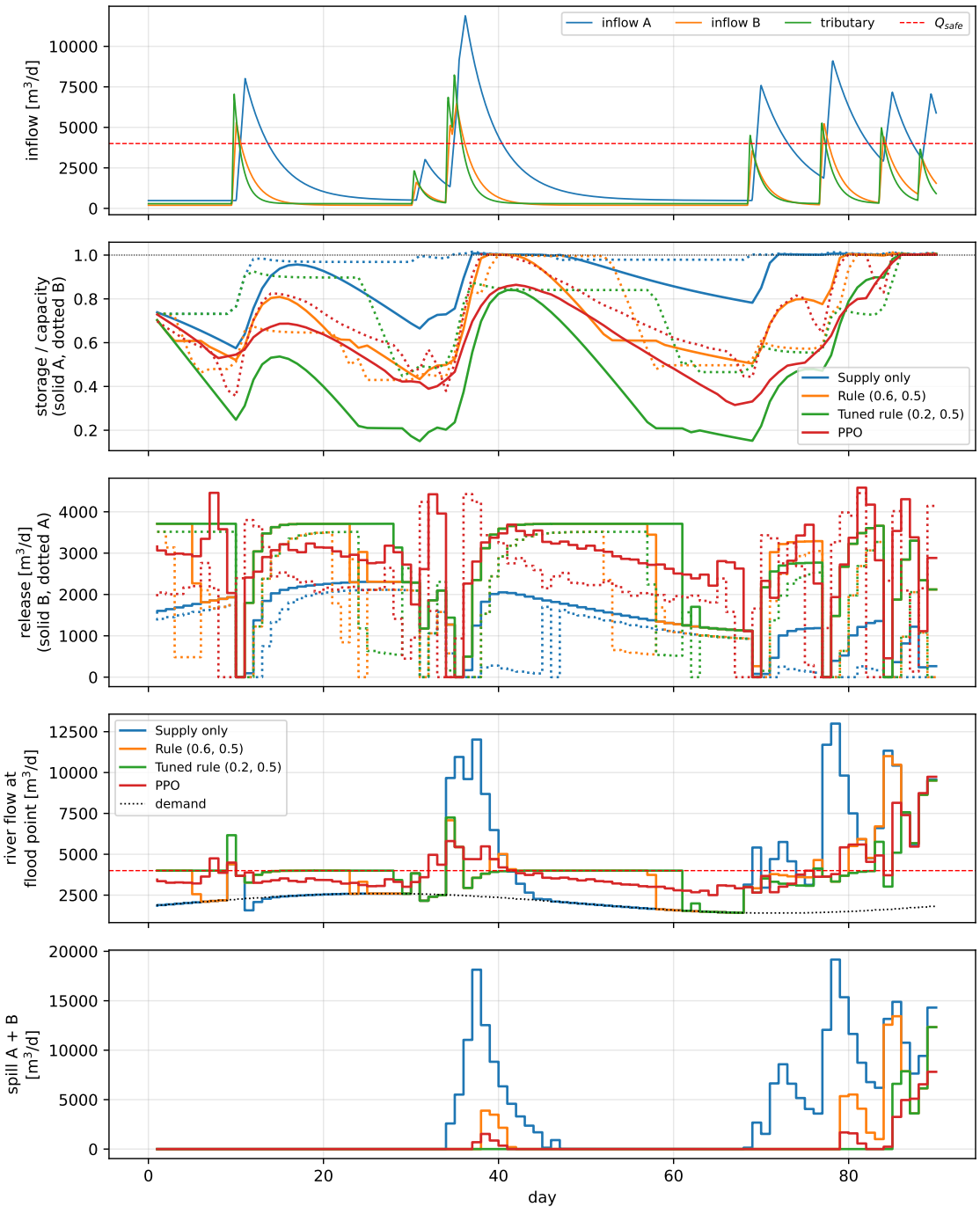


Figure 21: Cascade test season where PPO does worst relative to the tuned rule.

Holding water (valve nearly closed) makes it older, which is good for treatment. But the pond is then full when the next storm arrives and it overflows, which is bad for Q_p. Releasing keeps room but passes young, untreated water. A good operator holds water in dry weather and makes room before storms, which is a natural job for a forecast-aware learned policy.

10.1 System and data

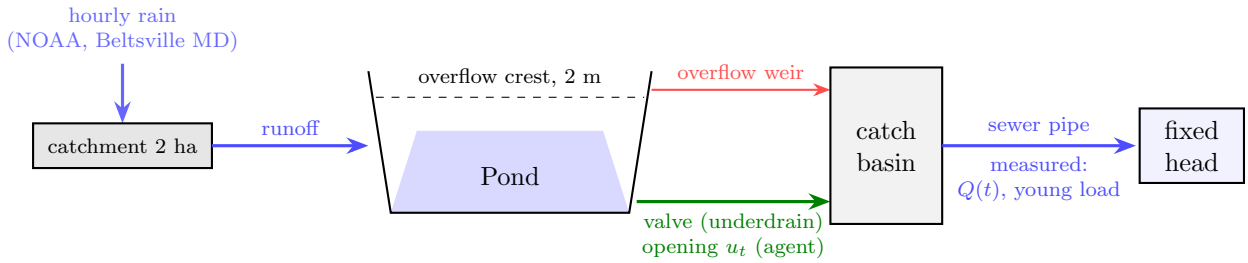


Figure 22: The stormwater pond. Both the valve (underdrain) and the overflow discharge into a catch basin, which drains to the receiving water (a fixed head) through a sewer pipe. Flow and young-water load are measured on that pipe.

Rainfall. The pond is driven by observed rainfall from NOAA's Hourly Precipitation Data network (HPD v2, NOAA NCEI): station USC00180700, Beltsville, Maryland, which lies between Baltimore and Washington DC. The record runs from 1948 to 2026 at hourly resolution. Values are converted from hundredths of an inch to meters, and flagged or missing hours are kept as gaps (`prepare_rain.py`). Annual totals of 1,000–1,200 mm, and a record 1,804 mm in 2018, match the regional climatology. Each episode is a **three-year** window (26,280 hours), as in the St. Francis study. A short window lets a controller exploit end effects (see "Closing loopholes" below), and Q_p at a small p needs a long record. No three-year window is entirely complete, so windows may have at most 1% missing hours, which are taken as dry. Training windows start anywhere in 1980–2019 where this holds (in practice 1996–2017 starts, about 12 distinct years of weather). The test windows are the two disjoint periods 2020–2022 and 2023–2025, which the agent never sees. Because two windows give weak statistics, the comparison is also made year by year (six test years).

Catch basin. Following the St. Francis model, the valve and the overflow weir both discharge into a `Catch basin` block (4 m^2, `Sewer_system.json`). It drains through a `Sewer_pipe` (0.3 m diameter, Manning $n = 0.01$, 10% slope; capacity about 34,000 m^3/d) to a fixed head. Measuring at the pipe means that overflow water is counted with the tracer concentration OHQ actually carries through the system.

Catchment and pond. Runoff is the rain on a 2-ha impervious catchment with runoff coefficient 0.9, entering the pond as a piecewise-constant hourly inflow. The pond is OHQ's `Pond` block (`Pond_Plugin.json`), whose storage–depth relation is approximately $S = \alpha h^{\beta}$, with $\alpha = 250$, $\beta = 2$. The overflow crest at $h = 2$ m corresponds to 1,000 m^3, about 55 mm of runoff from the catchment. An overflow weir (`wier`, $Q = 440{,}000\,(h-2)^{1.5}$ m^3/d) discharges anything above the crest.

The valve. The valve is a new OHQ link type, `controlled_orifice` (added to `resources/Pond_Plugin.json` during this work): the orifice law with the valve opening $u \in [0,1]$ as a property,

$$Q_{valve} = 86400\, C_d\, u\, A_{max} \sqrt{2g}\, \frac{h}{\sqrt{h+\varepsilon}}, \qquad C_d = 0.6,\ A_{max} = 0.03 \text{ m}^2,\ \varepsilon = 1 \text{ mm}.$$

The factor $h/\sqrt{h+\varepsilon}$ equals $\sqrt{h}$ for any real head but becomes linear near an empty pond. With the plain $\sqrt{h}$ law, whose slope is infinite at $h = 0$, the implicit solver failed whenever an open valve drained the pond within an hour. The St. Francis valve devices use the same regularization. While checking this link against a hand calculation, we found that OHQ currently evaluates a function of a *pure constant* as the constant itself: $\sqrt{19.6}$ written as `_sqr(19.6)` evaluated to 19.6, which made the valve 4.4× too large. The link type therefore writes $\sqrt{2g} = 4.427$ as a number, so that it also works with library builds that predate the fix of that bug (since committed to OHQ). Here h is the head difference between the pond and the catch basin.

10.2 Measuring water age with the Laplace tracer

OHQ's `AgeTracker_laplace` constituent, set up exactly as in the St. Francis residence-time study (`add_age.py`), carries a tracer at concentration 1 in every inflow. The tracer decays at a first-order rate λ wherever it is. A parcel of water that has spent time τ in the system therefore carries concentration $e^{-\lambda\tau}$. If the water in the pond (or leaving it) is a mixture of parcels with age density $p(\tau)$, its concentration is

$$C = \int_0^\infty p(\tau)\, e^{-\lambda\tau}\, d\tau = \mathcal{L}\{p\}(\lambda),$$

the Laplace transform of the age distribution evaluated at $s = \lambda$: hence the name. With $\lambda = 1$ d^{-1}, water younger than a few hours counts almost fully ($e^{-\lambda\tau} \approx 1$) and water older than a few days hardly at all. So C acts as a smooth "fraction of young water" with a time scale of about one day. The **young-water fraction** of the outflow over an episode is the exported tracer load divided by the outflow volume:

$$F_y = \frac{\int Q(t)\, C(t)\, dt}{\int Q(t)\, dt}.$$

10.3 OHQ model and checks

The Python binding cannot create constituents or reactions, so each hour's model is written as a small `.ohq` script and loaded with `create_from_script`. The script holds the templates, the tracer and its decay reaction, the pond (with the current storage and tracer concentration as initial conditions), the valve with this hour's opening, the weir and the outlet. It is solved with twelve 5-minute steps, again single-threaded. The pipe's hourly volume and young-water load are taken from the catch basin's balance: inflow from the pond (valve and weir fluxes at the end-of-step pond state, the valve seeing the basin's water level) minus the change in the basin's water and tracer content, minus tracer decay in the basin. This is exact by construction, whereas integrating OHQ's interpolated rate outputs left a 3% balance error. The pipe-flow samples from OHQ's output give the 5-minute flows used for Q_p. Checks:

- **Valve law:** the outflow implied by OHQ's storage drop over a step matches the formula above, evaluated at the end-of-step depth (backward Euler), to within 0.1%.
- **Tracer:** closed valve, $S_0 = 1{,}900$ m^3, $C_0 = 0.3$, inflow 2,000 m^3/d for one hour. A hand mass balance (tracer inflow +83.3, decay $\lambda M\, \Delta t \approx -23.8$) gives $C \approx 0.317$; OHQ gives 0.3168.

- **Water balance** over 800-hour test runs closes to within 0.01 m^3 out of 1,400–3,100 m^3 of inflow.

Making it fast enough. A typical hourly step costs 20–50 ms: about 23 ms to parse the script and templates, plus the solve. Three problems had to be fixed first:

1. **Inflow file.** OHQ re-reads the inflow time series at every build. A whole-episode file (52,000 points for three years) made each step take seconds, so each step now gets a two-point file for its own hour.
2. **Round-off states.** When the pond or catch basin empties, storages of 10^{-7}–10^{-11} m^3 and decayed tracer concentrations of 10^{-150} made OHQ's Newton iterations crawl, at 1–2 s per hour. Values below 0.1 m^3 (pond) and 0.01 m^3 (basin) are therefore set to exactly zero between steps.
3. **Lockstep rollouts.** Even then about 0.6% of hours take around a second. With 16 simulators advanced in lockstep, every step waited for the slowest, and throughput collapsed to about 20 steps/s. The trainer now lets each worker collect its rollout independently (Section 6).

At 20–50 ms per step, a three-year episode costs about 15–20 minutes of CPU, the clearest case so far for a compiled (codegen) backend.

10.4 Objectives, reward and the exceedance problem

Evaluation metrics. For each test window we compute, from the outflow at every 5-minute solver step:

- Q_p with $p = 0.001$: the flow exceeded 0.1% of the time (about 43 minutes per month), the value used in the St. Francis study;
- the young-water fraction F_y;
- a scalar objective used to tune baselines and rank policies, with each metric normalized by the *passive* pond (valve fully open) under the same rain:

$$J = \frac{Q_p}{Q_p^{passive}} + \frac{F_y}{F_y^{passive}}.$$

  Lower is better. The passive pond scores exactly 2, and halving either metric relative to doing nothing is worth the same. The normalization is done per test window and per test year. For the tuning grid it uses the passive pond's averages over the two tuning windows. Because any weighting is a judgment call, we also show the full trade-off between Q_p and F_y.

Closing loopholes. Earlier versions of this problem show how readily an optimizer exploits gaps in an objective:

- With a pond twice as large (2,000 m^3) and 30-day episodes, the best tuned rule kept the valve shut all month. A month's runoff fit in the pond, so almost nothing left and Q_p was near zero, and every window ended with a full pond, a cost the window never saw. A realistically sized pond (1,000 m^3) and three-year episodes remove this.
- Water still in the pond or catch basin at the end of an episode is counted as exported at its current tracer concentration, so holding water to the end cannot hide young water.

- With 30-day episodes, PPO learned to keep the pond near the crest and let it overflow in short bursts. In the first model the overflow went straight to the outlet and the age tracer treated it as old pond water. Routing both the valve and the overflow through the catch basin, and measuring there, makes OHQ carry the overflow's age consistently.

Following the design intent, there is no explicit penalty on overflow and no minimum-storage rule. Overflow matters only through its effect on the exceedance flow and on the young-water fraction, and the agent is free to use it whenever that helps.

Choosing p. We first used $p = 0.01$. Over a 30-day window that is about 7 hours, and a short overflow lasting less than that falls entirely inside the ignored 1%. A rule that kept the valve shut and let the pond overflow scored a Q_p of only 264 m^3/d despite overflow peaks of 8,800 m^3/d. With $p = 0.001$, Q_p reflects such peaks, and the training surrogate (which penalizes every high flow) and the evaluation metric agree.

Why not use Q_p as the reward directly? RL maximizes a *sum* of per-step rewards (Section 2.5). A quantile is not a sum: whether a particular hour's flow matters depends on all other hours of the episode. Its only exact form would be a single reward at the very end of the 720-hour episode, which gives the agent almost no signal about which hours mattered. We therefore train on a smooth *surrogate* that is additive, and evaluate on the exact Q_p:

$$r_t = -w_q \frac{1}{\Delta t} \sum_n \Big(\frac{\max(0,\, Q_n - Q_{ref})}{Q_{ref}} \Big)^2 \delta t \;-\; w_y \frac{\sum_n Q_n C_n \, \delta t}{V_{ref}},$$

summed over the solver steps n of hour t. The first term penalizes flows above Q_{ref}, and penalizes large ones much more because of the square, which pushes down the upper tail of the flow distribution, where Q_p lives. The second term is the exported young-water load, which is additive, so it represents F_y exactly up to the (nearly policy-independent) total outflow volume. Here $w_q = 1$, $w_y = 5$ and $V_{ref} = 1{,}000$ m^3. The weights roughly mirror J: with $Q_p^{passive} \approx 5{,}000$ m^3/d and $F_y^{passive} \approx 1$, $J \propto Q_p/1{,}000 + 5F_y$.

Observation and action. The agent observes 9 numbers: pond depth, pond tracer concentration C, current inflow, noisy rain forecasts for the next 1, 3, 6, 12 and 24 hours (multiplicative log-normal error growing with lead time), and the fraction of the window left. It sets the valve opening once per hour. The network's action $a_t \in [-1, 1]$ is mapped to the opening as $u_t = ((a_t + 1)/2)^2$. With the linear mapping used at first, the useful openings of 0–10% occupied only the bottom 5% of the action range; with the square they occupy about a third of it.

10.5 Baselines

- **Open valve** ($u = 1$): a conventional dry pond with a fixed orifice.
- **Fixed opening** u: a throttled orifice; u tuned over $\{0.03, 0.05, 0.1, 0.2, 1\}$.
- **Forecast rule**, in the spirit of the St. Francis forecast-informed control: hold at a small opening u_{hold} so that water ages; when the 24-hour rain forecast exceeds a threshold, pre-release at u_{pre}; open fully if the depth exceeds 1.8 m. The three parameters are tuned by grid search over 24 combinations ($u_{hold} \in \{0, 0.02\}$, $u_{pre} \in \{0.05, 0.1, 0.2, 0.4\}$, threshold $\in \{10, 15, 30\}$ mm), centred on the best settings found with 30-day windows.

Both rule families are tuned on two three-year training-period windows (starting 1996 and 2009); the results are fixed opening $u = 0.02$; forecast rule $u_{hold} = 0$, $u_{pre} = 0.1$, threshold 15 mm .

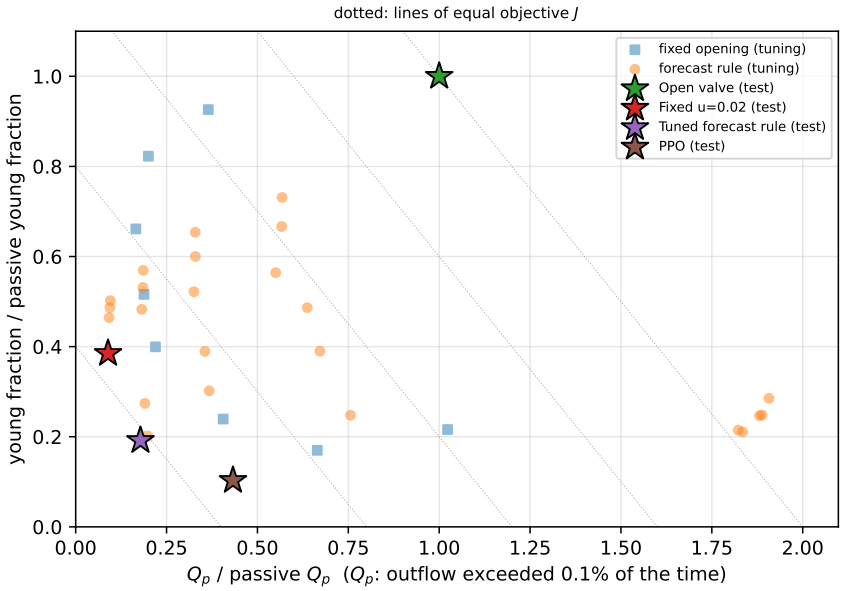


Figure 23: Trade-off between peak-flow control (Q_p) and treatment (young fraction). Small symbols: every baseline setting tried during tuning (training windows). Stars: the four compared policies on the test windows. Dotted lines connect points with equal objective J.

10.6 Training

The first attempt trained PPO from scratch with a linear action mapping and raw rewards. It converged to a poor policy that opened the valve only when the pond was already empty, kept the pond at the crest, and scored a normalized J of 1.13–1.23 on the test windows, barely better than the passive pond's 2. The run evaluated in Tables 6 and 7 uses the three remedies described in Section 6: the square action mapping, reward normalization, and a warm start by behavior cloning of the tuned forecast rule. The cloned actor reproduced the rule's 48,000 decisions to within 0.008 on the action scale. PPO then trained for 1.5 million hourly steps, about 2.5 hours on a machine shared with other jobs. Its first rollouts already scored at the rule's level (about -0.005 per hour, versus -0.10 from scratch). The raw reward per hour kept improving to about -0.0004 by the end, and the exploration spread fell from 0.30 to 0.13. A second warm-started run without reward clipping, with snapshot selection by J, is described at the end of the results.

10.7 Results

Headline. Over the full three-year test windows, the tuned forecast rule is the best policy ($J =$ 0.375 and 0.365, i.e. about 82% better than the passive pond). Next come the 2% fixed opening (0.498, 0.450) and PPO (0.588, 0.483). Year by year, PPO and the rule are statistically tied ($\Delta J = -0.08 \pm 0.22$, PPO better in 2 of 6 years). The three-year metric, the St. Francis definition, favors the rule because PPO's overflows cluster in a few years and raise the three-year Q_p. The warm start made a large difference: from scratch, PPO scored $J \approx 1.1$–1.2.

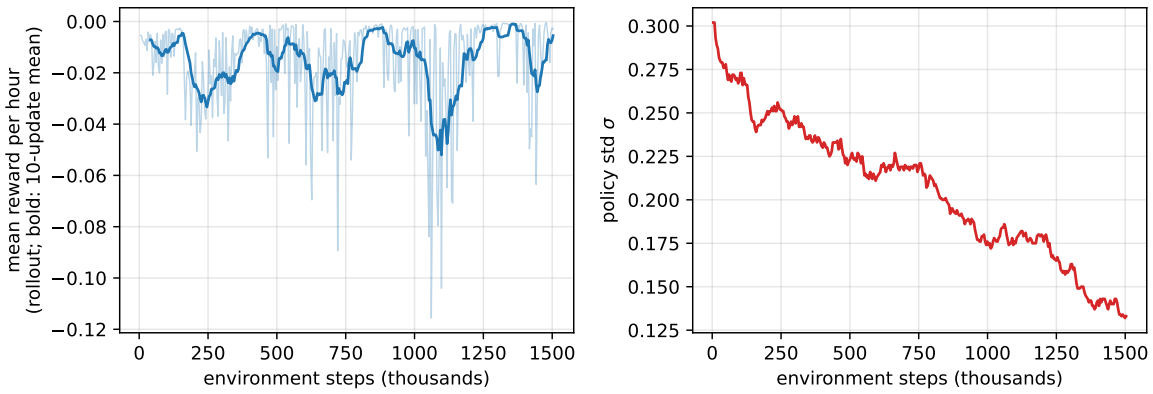


Figure 24: Pond training: mean return of the last 20 episodes (left), policy standard deviation (right).

Policy	window	J	Q_p [m^3/d]	young fraction	peak [m^3/d]	overflow [m^3]
Open valve	2020–22	2.00	5182	0.993	8391	0
	2023–25	2.00	4971	0.993	8558	0
Fixed u=0.02	2020–22	0.50	697	0.361	11986	2349
	2023–25	0.45	216	0.403	11525	1384
Tuned forecast rule	2020–22	0.37	912	0.197	12056	162
	2023–25	0.36	896	0.183	11111	85
PPO	2020–22	0.59	2487	0.107	12309	11819
	2023–25	0.48	1917	0.097	12195	6277

Table 6: Pond: results for the two three-year test windows. J is the evaluation objective (lower is better).

What PPO changed relative to the rule. PPO started as an exact copy of the rule and moved toward holding more water. Its outflow is the oldest of all policies (young fraction 0.10–0.11, versus 0.18–0.20 for the rule). But it ends both windows at the crest and overflows 6,000–12,000 m^3 per window (the rule: 85–162 m^3). Its Q_p is therefore 2–3 times the rule's. In terms of J, the gain in age does not pay for the loss in peak-flow control.

Why: the training reward is not exactly J. During training PPO kept improving the reward it was given: the raw reward per hour improved from -0.028 to -0.0004. That reward differs from J in two ways, and both undervalue overflow:

1. the peak term is a smooth surrogate of the exceedance quantile, not the quantile itself;
2. reward normalization *clips* rewards at ± 10 standard deviations. That stabilized learning, but it also caps the penalty of exactly the rare, extreme overflow hours that determine Q_p.

So PPO optimized the stated reward well, but the stated reward traded overflow for age at a different rate than J does.

What would likely close the gap.

- Normalize rewards without clipping (or with a much wider clip), so the overflow penalty keeps its weight.

baseline	mean ΔJ (per year)	std. error	PPO better in	mean ΔQ_p / Δyoung
Open valve	-1.494	0.054	6/6 years	-3031 / -0.892
Fixed u=0.02	$+0.031$	0.050	2/6 years	$+1616$ / -0.281
Tuned forecast rule	-0.078	0.216	2/6 years	-12 / -0.088

Table 7: Pond: paired differences, PPO minus baseline, computed for each of the six test years. Negative ΔJ means PPO is better.

- Select the policy by J: evaluate checkpoints during training on held-out training-period windows and keep the best one, rather than the last.
- Keep the policy close to the rule unless it clearly improves on it, e.g. by penalizing deviation from the cloned rule early in training.

Lessons. Two lessons stand out. First, starting from an engineering rule turned an RL failure into a policy competitive with a carefully tuned rule. Second, when the true objective (a quantile over three years) cannot be the reward, every simplification of it, including the numerical stabilizations, becomes part of the objective the agent actually optimizes. Whether the result is better must be judged on the true objective, with tuned baselines, on data the agent has not seen.

Retraining with both fixes. PPO was retrained from the cloned rule, with reward normalization but *no clipping*, saving a snapshot every 61 updates. Each snapshot was then scored by the true objective J on the two three-year validation windows: the training-period windows starting in 1996 and 2009, also used to tune the baselines. Holding these windows out of training would have left only about 4–7 years of training weather, because the gauge record has gaps. The test windows remained untouched.

snapshot (update)	steps	J (validation)	Q_p [m^3/d]	young fraction
0 (cloned rule)	0	0.427	1042 / 1101	0.190 / 0.198
61	249856	0.764	2525 / 2923	0.166 / 0.182
122	499712	0.555	1862 / 2140	0.108 / 0.136
183	749568	0.737	2444 / 2898	0.149 / 0.167
244	999424	0.703	2289 / 2810	0.140 / 0.162
305	1249280	0.480	1167 / 2165	0.106 / 0.138
367 (last)	1503232	0.596	1699 / 2742	0.102 / 0.133

The answer is unambiguous: the best snapshot is the starting one, i.e. the cloned rule ($J = 0.427$ against 0.400 for the rule itself on these windows). *Every* PPO update made J worse (0.48–0.76), even though the training reward kept improving (the last completed episodes scored -244, against -461 in the clipped run). Selection therefore returns the rule (its clone), and a test evaluation would reproduce the rule's results. Removing the clipping was not enough. The problem is the training reward itself: it values a lower young fraction more, relative to Q_p, than J does. Every snapshot buys older outflow (young fraction 0.10–0.14 instead of about 0.19) at the price of roughly doubling Q_p, which is a bad trade in terms of J.

Next step: make the reward agree with J. The reward weights ($w_q = 1$ on the squared excess above Q_{ref}, $w_y = 5$ on the young load) were set by a rough scale argument. A principled alternative is to calibrate them: for the tuned baseline settings, whose J is known, compute their

reward as well, and choose w_q, w_y and Q_{ref} so that reward and J rank the settings the same way. Alternatively, make the reward match J by construction, e.g. with a terminal reward equal to $-J$ for each year of the episode. With both fixes in place, snapshot selection by J guarantees that the result can be no worse than the rule.

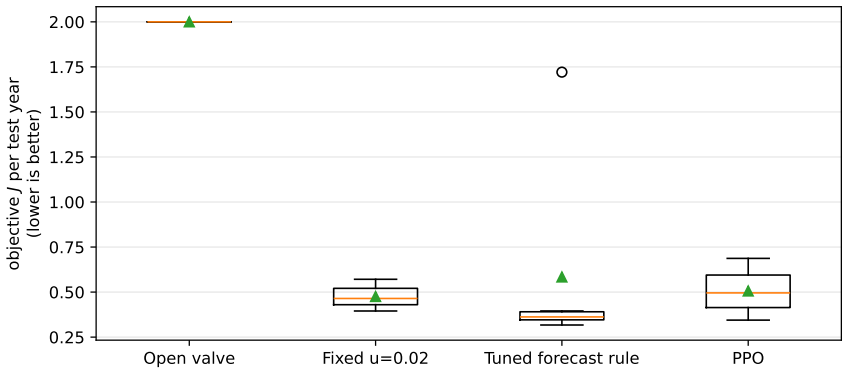


Figure 25: Pond: distribution of the objective J over the six test years.

11 Limitations and next steps

- **Make the problem harder than one rule can handle.** Candidates are a cascade of reservoirs, a water-quality objective (OHQ's constituent transport), hydropower with time-varying prices, or a longer, skill-varying forecast. These are the settings where RL should pull ahead of tuned rules.
- **Simulator speed.** Replace `OHQBackend` with a codegen-compiled kernel (Section 8). Even within the current bindings, a method to reset a block's state on a persistent `System` and an in-memory output accessor would remove most of the per-step overhead.
- **OHQ fixes uncovered here.** (i) *Done:* rebuilt `libOHQLib` so the bindings use the end-of-step output stamping. (ii) *Open:* move the `"unit"` entry out of the `rule` dictionary of `Reservoir_link_rule` in `main_components.json`. (iii) *Done:* `SetSystemSettingsObjectProperties` returned `false` whenever `settings.json` had not been loaded, even though the value had been applied. It now returns `SetProperty`'s result (branch `fix/set-system-value-return`), and the environment raises an error if OHQ rejects a setting.
- **RL details.** Training used a single seed and a fixed budget. Multiple seeds, observation and reward normalization, and a tanh-squashed policy would make the agent more robust. Daily-average flood accounting also underweights sub-daily peaks; the reward could use the peak river flow instead.

12 How to reproduce

```
source ~/Projects/ohq_rl_env/bin/activate
cd ~/Projects/OpenHydroQual-RL
python diagnostics.py # verification + timing figures
```

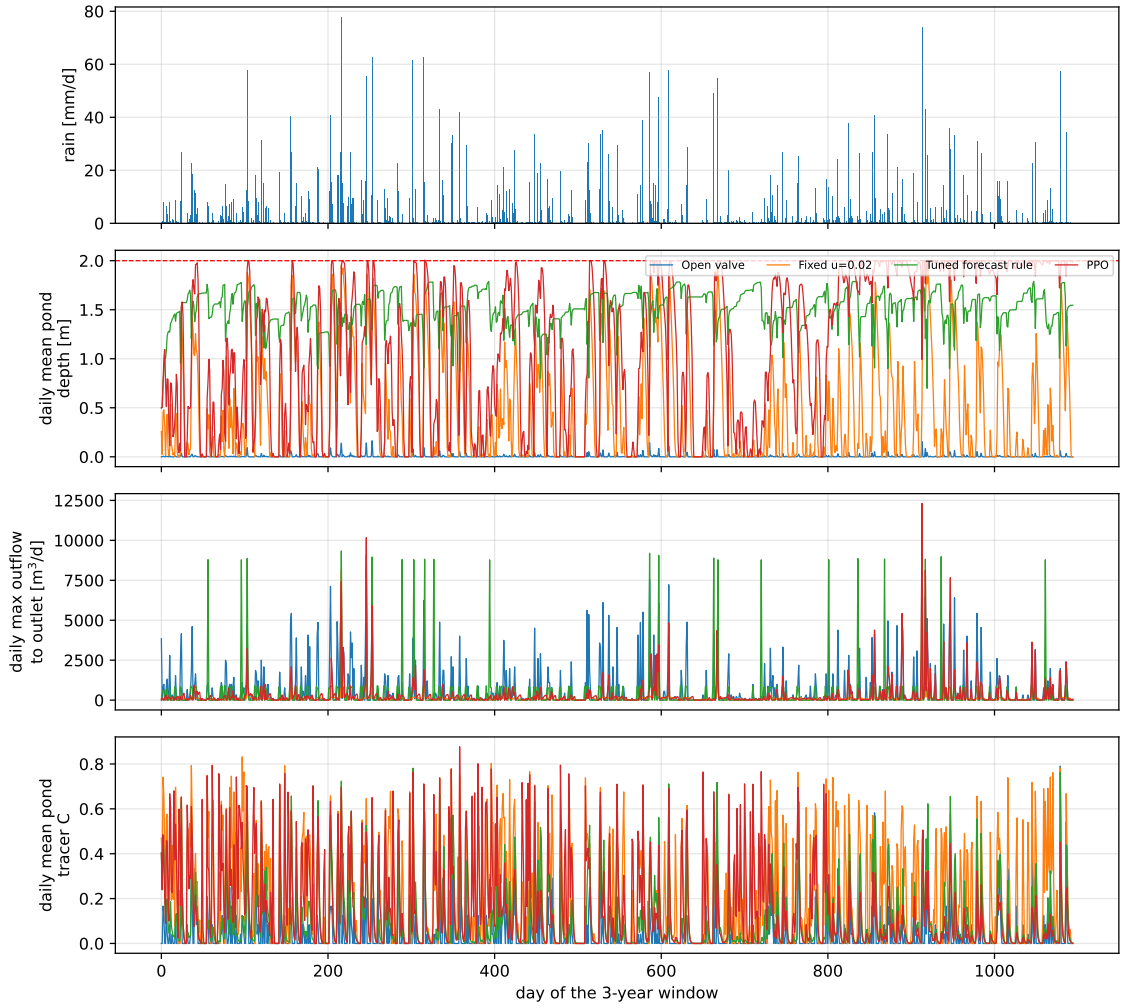


Figure 26: Test window 2020–2022, daily values. From top: daily rain; daily mean pond depth (dashed: overflow crest); daily maximum flow to the outlet; daily mean pond tracer concentration (lower means older water).

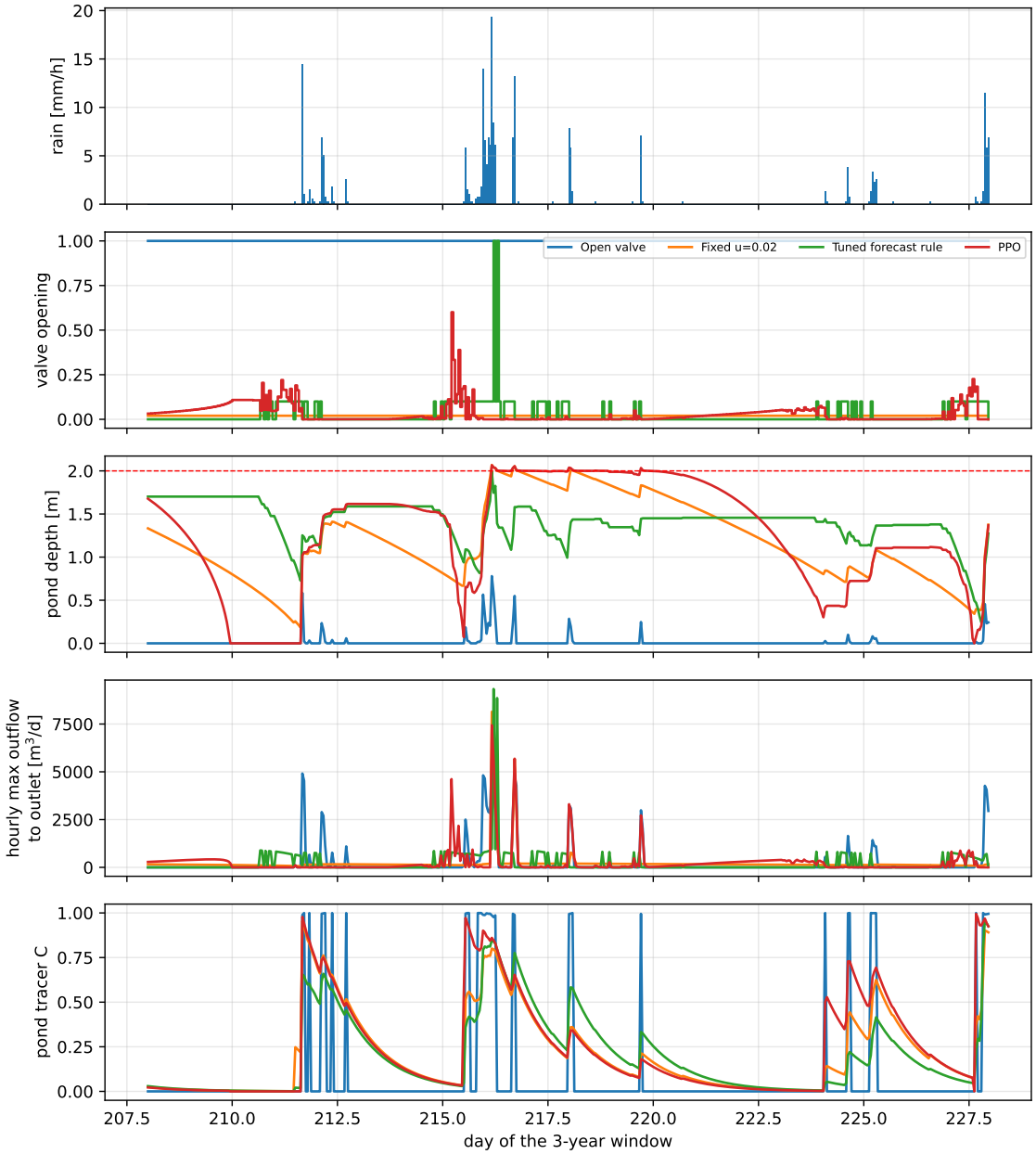


Figure 27: Test window 2020–2022, hourly detail of the 20 days around the wettest day. From top: rain; valve opening; pond depth; hourly maximum flow to the outlet; pond tracer concentration.

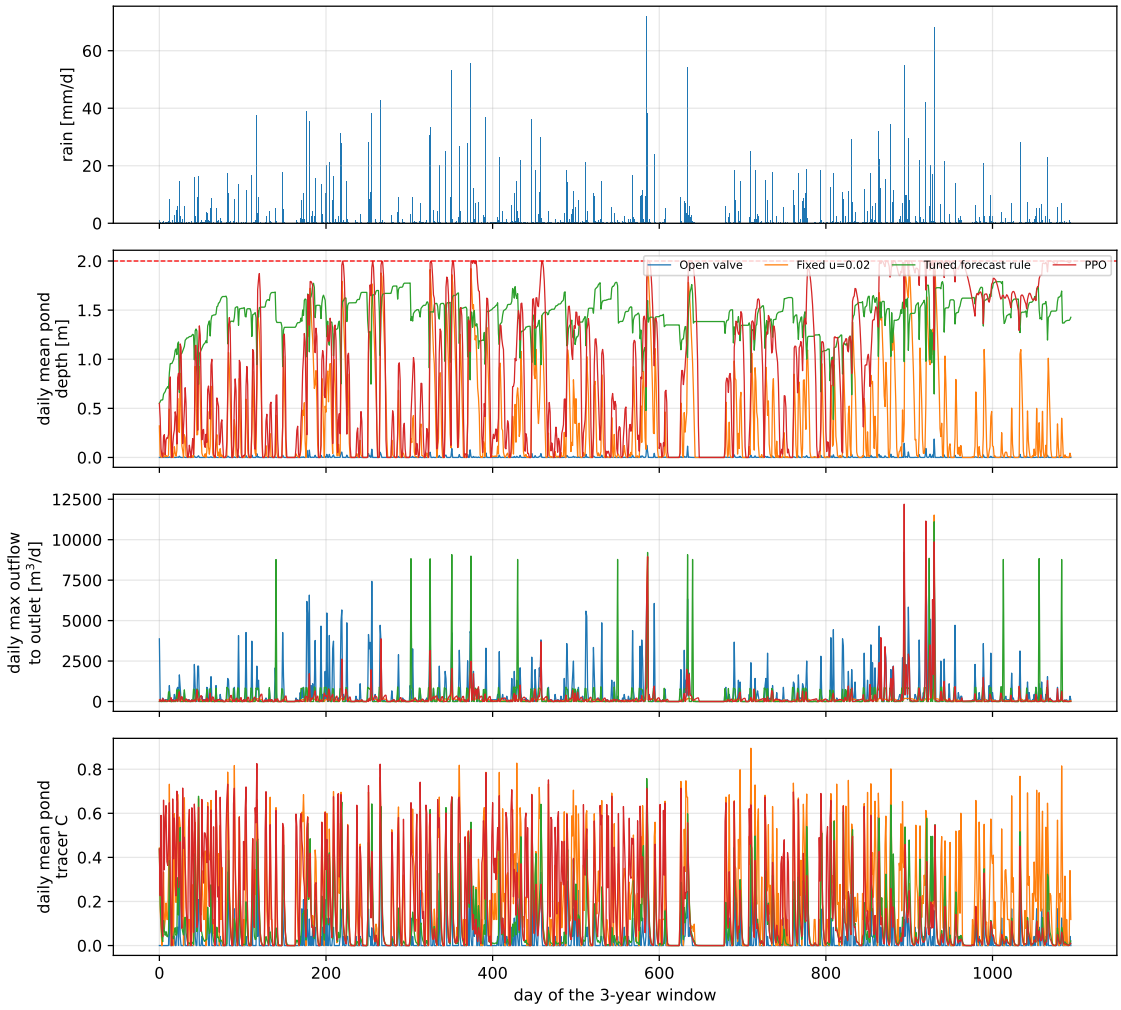


Figure 28: Test window 2023–2025, daily values (panels as in Figure 26).

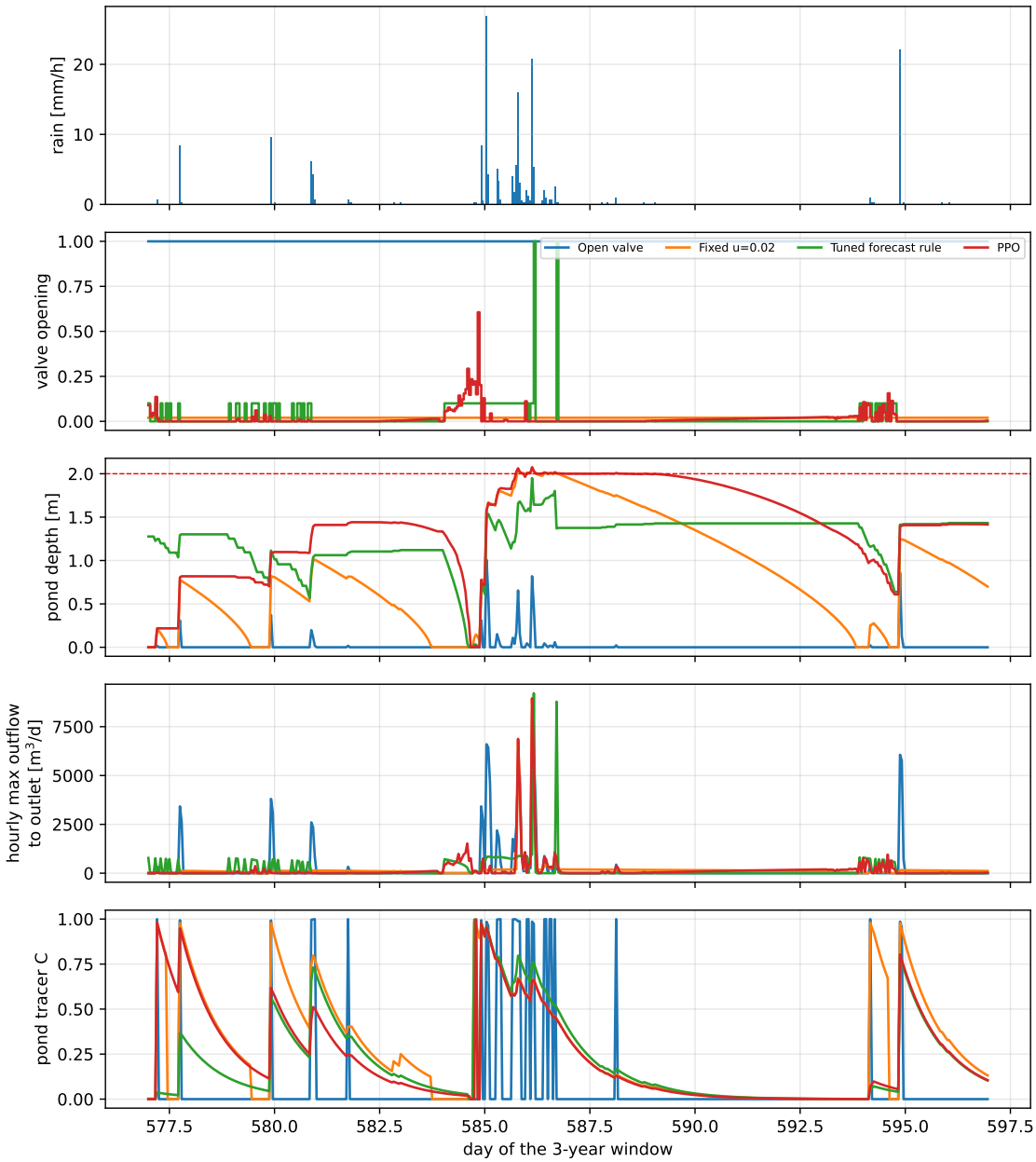


Figure 29: Test window 2023–2025, hourly detail around the wettest day.

```
python train_ppo.py --total-steps 409600 --num-envs 16 --out results/ppo_reservoir.pt
python evaluate.py --policy results/ppo_reservoir.pt --episodes 30
cd report && latexmk -pdf main.tex
```

The paths of the bindings and templates can be overridden with the `OHQ_PY_PATH` and `OHQ_RESOURCES` environment variables.

Appendix: RL glossary

Action (a_t) What the agent decides at a step; here the release(s) for the day.

Actor The network that represents the policy, $o \mapsto \mu_\theta(o)$.

Actor–critic A method with both a policy network (actor) and a value network (critic). PPO is one.

Advantage (A, $\hat{A}_t$) How much better an action turned out than the policy's average in that situation; positive means "do this more".

Agent The decision maker; here the learned operating rule.

Behavior cloning Supervised training of the actor to imitate an existing policy (here the tuned rule) from its recorded observations and actions; used as a warm start for PPO.

Baseline (i) In this report: a comparison policy (demand only, rule curves). (ii) In RL theory: a function subtracted in the policy gradient to reduce noise; the critic plays this role.

Bellman equation The recursion "value now = expected reward now + discounted value next" (Eq. 2); the backbone of both SDP and RL.

Bootstrapping Building a value label partly from the critic's own prediction of the future, instead of waiting for all actual rewards.

Clipping (PPO) Limiting the change in action probabilities per update to $\pm\epsilon$ (20%) so that learning is stable.

Credit assignment Working out which earlier actions caused a later reward or penalty.

Critic The value-function network $V_\phi(o)$ that predicts the return still to come.

Discount factor (γ) Weight < 1 applied per step to future rewards; sets the effective planning horizon $\sim 1/(1-\gamma)$ steps.

Environment Everything outside the agent: here the OHQ model plus the random storms and demand.

Episode One run from start to end; here one 90-day season.

Epoch One pass over the batch of collected data during an update.

Exploration Trying actions other than the current best guess, here through random noise around the mean action, to discover better behavior.

GAE Generalized advantage estimation: a weighted blend of multi-step TD errors (Eq. 10).

Generalization Performing well on situations (seasons) not seen in training; tested on held-out seeds.

Gymnasium The standard Python interface for RL environments (`reset`, `step`).

Hyperparameter A setting of the learning algorithm itself (learning rate, γ, ...), as opposed to the learned weights.

Learning curve Return versus amount of training experience.

MDP Markov decision process: states, actions, transition law, rewards; the mathematical model of a sequential decision problem.

Minibatch The subset of records used for one gradient step.

Neural network A flexible parametric function built from layers of simple nonlinear units; its weights are fitted by gradient descent.

Observation (o_t) The information the agent receives at a step; may be less than the full state.

On-policy Learning only from data collected by the current policy (PPO is on-policy, so it discards data after each update).

Off-policy Learning from data collected by older or other policies, typically stored in a replay memory (e.g. DQN, SAC); PPO is not off-policy.

Policy (π) The decision rule mapping observations to actions; stochastic during training, deterministic in evaluation.

Policy gradient The gradient of expected return with respect to the policy parameters (Eq. 3); estimated from simulations without differentiating the simulator.

PPO Proximal Policy Optimization: the policy-gradient actor–critic algorithm used here.

Replay memory A store of past experience reused for many updates by off-policy methods; PPO has none and discards each batch after one update.

Return Sum of (possibly discounted) rewards over an episode; the quantity to maximize.

Reward (r_t) Scalar score of one step; its design defines the objective.

Rollout A stretch of simulated experience collected with the current policy.

Seed A number that fixes a random sequence; used to generate reproducible, distinct seasons.

State (s_t) Everything that determines the future evolution of the system.

TD error (δ_t) One-step surprise: $r_t + \gamma V(o_{t+1}) - V(o_t)$.

Value function (V^π) Expected return still to come from a state when following policy π.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.

[2] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel. High-Dimensional Continuous Control Using Generalized Advantage Estimation. ICLR, 2016.